Rust — A Working Book

A visual, example-driven tour of ownership, traits, and fearless concurrency

Jonghwan Lee

2026

Rust — A Working Book

Contents

Preface 6

Who this book is for . . . 6
What this book is not . . . 6
How to read the book . . . 7
A map of the book . . . 7
Conventions . . . 8
A personal note . . . 9

Why Rust 10

1.1 The three pillars . . . 10
1.2 The landscape . . . 12
1.3 A first program . . . 13
1.4 Why not just use C++? . . . 15
1.5 Why not just use Go? . . . 15
1.6 What you give up . . . 16
1.7 A look ahead . . . 16

Environment 17

2.1 The toolchain . . . 17
2.2 Installing Rust . . . 19
2.3 Hello, cargo . . . 19
2.4 Adding a dependency . . . 21
2.5 Tests . . . 21
2.6 Formatting and linting . . . 23

Rust — A Working Book

2.7 Editors 23
2.8 The workspace 24
2.9 A 30-second loop 25

Core Language **25**

3.1 Variables 25
3.2 Primitive types 26
3.3 Compound types 27
3.4 Control flow 28
3.5 Functions 30
3.6 Blocks are expressions 31
3.7 Printing and formatting 32
3.8 Modules and visibility 32
3.9 Standard output and input 33
3.10 A full small program 34

Ownership, Borrowing, Lifetimes **35**

4.1 Stack and heap 35
4.2 Ownership 37
4.3 Move semantics 37
4.4 Scope and drop 39
4.5 Borrowing 40
4.6 Lifetimes 43
4.7 A worked example 46
4.8 Reading compiler messages 46
4.9 Summary 47

Rust — A Working Book

Types: Structs, Enums, Traits, Generics 48

5.1 Structs . . . 48
5.2 Enums . . . 51
5.3 Generics . . . 53
5.4 Traits . . . 55
5.5 Worked example — a small parser error . . . 60

Error Handling 62

6.1 Two kinds of failure . . . 63
6.2 Option . . . 63
6.3 Result . . . 65
6.4 The `?` operator . . . 66
6.5 Custom errors . . . 67
6.6 `anyhow` for application code . . . 68
6.7 When to panic . . . 69
6.8 Catching panics . . . 70
6.9 Main can return a Result . . . 71
6.10 A complete picture . . . 71

Collections and Iterators 73

7.1 Vec . . . 73
7.2 String . . . 74
7.3 HashMap and HashSet . . . 75
7.4 BTreeMap and BTreeSet . . . 76
7.5 The Iterator trait . . . 77
7.6 Adapters and consumers . . . 78
7.7 Why iterators are zero-cost . . . 80

Rust — A Working Book

7.8 Closures 80
7.9 A small pipeline 81

Concurrency and Async **83**

8.1 Spawning threads 83
8.2 Send and Sync 85
8.3 Sharing data — Arc and Mutex 86
8.4 Channels 88
8.5 Atomics 89
8.6 Rayon — effortless data parallelism 89
8.7 Async — when blocking is the problem 90
8.8 Threads vs async 92
8.9 What you get 93

CHAPTER

Preface

Rust is a language that makes promises most languages do not even try to make. It promises memory safety without a garbage collector. It promises zero-cost abstractions, where high-level code compiles to the same instructions a careful C programmer would write by hand. It promises *fearless concurrency*, where the type system itself rules out whole categories of data races.

Those are extraordinary claims. They are also, in a very real sense, true. The price is a compiler that demands more of you at the moment you write the code, in exchange for fewer surprises at the moment it runs. This book is about paying that price cheerfully, and getting the promises back.

Who this book is for

You are the right reader for this book if you already know *a* language — any language — well enough to reach for it when you need to build something. Python, JavaScript, Go, Java, C#, C++, C, Swift; any of these will do. You do not need to know systems programming. You do not need to have fought a segfault. You do, however, need to be comfortable reading code carefully, because Rust rewards careful reading more than most languages do.

If you are coming from a garbage-collected language, you will spend the most time on **ownership** (Chapter 4). If you are coming from C or C++, you will spend the most time on **traits and lifetimes** (Chapters 4 and 5). Either way, plan for the slow middle chapters to be where the book happens.

What this book is not

This book is not a reference. It does not enumerate every feature. It does not cover `macro_rules!`, `proc-macro`, the Rust ABI, or the hairier corners of `unsafe`. It does not teach you embedded Rust or kernel Rust. What it does is teach you the shape of the language well enough that *The Rust Reference* and *The Rustonomicon* become books you can open and learn from on your own.

How to read the book

Read linearly. The chapters are ordered by dependency — each one uses ideas from the ones before it — and most chapters end with a short set of exercises that take fifteen minutes to an hour. You do not need to do every exercise, but try one per chapter. Rust is a language where reading code passively is deceptive; the compiler is a teacher you only meet by writing.

Every example in the book is a complete program or a fragment you can drop into `fn main() { … }`. If a snippet does not compile, that is usually the point — the surrounding text will show the error message the compiler produces and then fix it. Learn to read these messages. Rust's error messages are one of its best features, and a fluent Rust programmer is, in part, a fluent *reader of rustc's output*.

A map of the book

#

Chapter

What you will learn

0

Preface

Why this book, how to read it

1

Why Rust

The three pillars and where Rust sits in the language landscape

2

Environment

rustup, cargo, rustfmt, clippy, rust-analyzer

3

Core Language

Values, types, control flow, functions, expressions

4

Ownership, Borrowing, Lifetimes

The single most important chapter

5

Types: Structs, Enums, Traits, Generics

Building abstractions that cost nothing

6

Error Handling

`Option`, `Result`, the `?` operator, custom errors

7

Collections and Iterators

`Vec`, `HashMap`, lazy pipelines

8

Concurrency and Async

Threads, `Send` / `Sync`, `async` / `await`

Chapter 4 is the gate. You can skim everything after it and come back later, but you cannot skim Chapter 4. Every idea that follows — every trait bound, every lifetime annotation, every `Arc<Mutex<T>>` in Chapter 8 — assumes you have internalised what ownership means.

Conventions

Runnable code blocks use the `rust` tag:

```rust
fn main() {
    println!("Hello, world!");
}
```

Compiler output — what `rustc` or `cargo build` prints — uses the `text` tag, and we quote it verbatim so you can recognise messages when you meet them in the wild:

```
error[E0382]: borrow of moved value: `s`
  --> src/main.rs:4:20
   |
 2 |     let s = String::from("hi");
   |         - move occurs because `s` has type `String`
 3 |     let t = s;
   |             - value moved here
 4 |     println!("{s}");
   |                ^ value borrowed here after move
```

When an example *must not* compile, the heading says so. When an example *should* compile and run, you should be able to paste it into a fresh `cargo new` project and see it do so.

Three kinds of callouts appear throughout the book:

> **Note**
>
> **Note.** A digression or clarification. Safe to skip on a first pass.

> **Tip**
>
> **Tip.** A practical shortcut — something that will make your life better once you trust it.

> **Warning**
>
> **Warning.** A sharp edge. Read these twice.

A personal note

Rust is a language that feels slow to learn and fast to write. The first week is uphill. The second week is uphill with a heavier pack. By the third week, the compiler stops arguing with you — not because it has changed, but because you have, and now you are writing the code it was going to demand anyway. The switch is not gradual; it is sudden, and it arrives around the time you stop thinking of lifetimes as annotations and start thinking of them as *facts about your program*.

That moment is the whole point of this book. Turn the page.

Next: Chapter 1 — Why Rust

CHAPTER

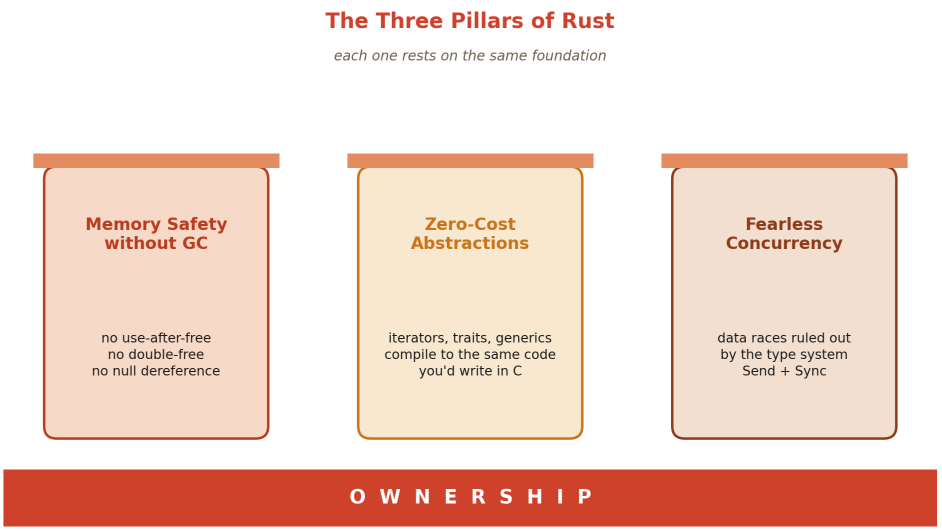
Why Rust

Every language is a bet about what matters. C bets that giving you total control is worth the cost of doing the bookkeeping yourself. Python bets that programmer ergonomics are worth giving up on raw speed. Go bets that a garbage collector and a small standard library are enough for most server-shaped problems, and that simplicity beats cleverness every time.

Rust's bet is different. Rust bets that a sufficiently clever type system can give you **both** the control of C and the safety of a managed language, without a garbage collector, and at zero runtime cost. That is a larger bet, and it took ten years of careful engineering to make it pay off. When people say Rust is difficult, they are usually describing the experience of meeting that type system for the first time. When people say Rust is *worth it*, they are describing what happens three months later, when you start writing code that does not crash.

This chapter is the one-page version of that argument. We will lay out the three pillars, show where Rust sits relative to other languages, and write a short program that demonstrates, in miniature, what the rest of the book is about.

1.1 The three pillars



Rust's three design commitments. Each rests on the ownership system — remove ownership, and the other two collapse.

Rust's identity rests on three claims:

Memory safety without a garbage collector. Your program cannot use memory after it is freed, cannot free it twice, cannot read uninitialised memory, and cannot produce a null pointer dereference. No runtime watches over you. The checks happen at compile time, via the *borrow checker*, which we meet in Chapter 4.

Zero-cost abstractions. You can write high-level code — iterator chains, generic containers, trait objects — and the compiler will turn it into assembly that is as good as what you would write by hand in C. Bjarne Stroustrup's line about C++ applies more honestly here: *what you do not use, you do not pay for, and what you do use, you could not write any better yourself.*

Fearless concurrency. The same type system that prevents use-after-free also prevents data races. Sharing data between threads requires the data's type to implement certain marker traits (`Send` and `Sync`, Chapter 8) — and if a type does not satisfy them, the compiler rejects the code before it ever runs. You can still deadlock, you can still write slow concurrent code, but the category of bug that has haunted systems programming for forty years — *two threads touching the same memory without synchronisation* — is simply gone.

All three of these rest on a single idea: **ownership**. Every value in a Rust program has exactly one owner; when the owner goes out of scope, the value is dropped; references to the value are checked by the compiler to make sure they never outlive it. That idea sounds modest. It is not. Chapter 4 is about why.

1.2 The landscape

Where does Rust sit relative to the languages you already know? The axes that matter are *what runtime cost you pay* and *what safety guarantees you get in return*.

Language

Runtime cost

Memory safety

Data-race safety

Manual memory

C

None

No

No

Yes

C++

None

Mostly no

No

Yes (usually)

Go

GC + runtime

Yes

No

No

Java/C#

GC + JIT

Yes

No

No

Python

Interpreter

Yes

(GIL, sort of)

No

Rust

None

Yes

Yes

No***

The asterisk: Rust does not ask you to call `free`. It asks you to write code whose *structure* tells the compiler when to call `free` for you. The compiler inserts the `drop` calls; you do not pay for a garbage collector; you do not risk forgetting.

This is the unusual position Rust occupies. It is as fast as C, as safe as Go, and it asks for something in return that neither of them asks for: a more careful relationship with who owns what.

1.3 A first program

Here is a Rust program that does something small and real. It reads a file of numbers, one per line, and prints the sum and the mean.

```rust
use std::fs;
use std::io;

fn main() -> io::Result<()> {
    let text = fs::read_to_string("numbers.txt")?;

    let nums: Vec<f64> = text
        .lines()
        .filter(|l| !l.trim().is_empty())
        .map(|l| l.trim().parse().expect("not a number"))
        .collect();

    let sum: f64 = nums.iter().sum();
    let mean = sum / nums.len() as f64;

    println!("count = {}", nums.len());
    println!("sum   = {sum:.3}");
    println!("mean  = {mean:.3}");

    Ok(())
}
```

A lot is happening in twelve lines. Count the things Rust is doing for you:

1. `?` **at the end of** `read_to_string("numbers.txt")?` **.** If the file is missing, the function returns the error. No exception, no null check — the error is a value, and `?` is the shorthand for *propagate-if-error*. Chapter 6 is about this.
2. **The iterator chain** `.lines().filter(…).map(…).collect()` **.** Zero allocations until the last step. `lines()` and `filter` and `map` are *lazy*; `collect()` is the only step that touches the heap. Chapter 7 is about why this is fast.
3. **The type annotation** `let nums: Vec<f64>` **.** That annotation is what tells `collect()` which container to produce. `collect()` is generic over its return type; the annotation is how you pick. Chapter 5 is about the trait system that makes this possible.
4. `io::Result<()>` **as the return type of** `main` **.** Errors are just values. `main` can return one. If it does, the program exits with a non-zero status and prints the error. No magic.

If you know any language in the table above, you can probably read this program at a glance. What is harder is the question of *why this compiles* and *what guarantees you get* — and that is the rest of the book.

1.4 Why not just use C++?

A fair question, especially if you already know C++. The short answer: C++ gives you the tools to write safe code, but it does not make you use them. Rust makes you use them. That is the whole difference.

A slightly longer answer:

- In C++, a dangling reference is a runtime bug. In Rust, it is a compile error.
- In C++, a data race is a runtime bug and often an undefined one. In Rust, it is a compile error.
- In C++, whether a value is moved, copied, or aliased is often implicit and version-dependent. In Rust, it is in the type.
- In C++, memory ownership is convention: smart pointers, RAII, the rule of five. In Rust, it is the language.

C++ gives you ten ways to do the right thing. Rust gives you one, and it is checked.

> **Note**
> This is not an argument that Rust is *better*. Enormous, important systems are written in C++ and will be for decades. But if you are starting new systems code today, and you do not have a specific reason to pick C++ (existing codebase, toolchain constraints, C++-only libraries), Rust is almost always the safer and more productive choice.

1.5 Why not just use Go?

Another fair question, aimed at a different problem. Go and Rust overlap in the "modern, compiled, server-backend" space, and people sometimes frame the choice as a shootout. It is not, because they are solving different problems.

Go chose *simplicity and a garbage collector*. You get a small language that a team can learn in a week, a runtime that handles memory, and goroutines that make concurrency ergonomic. You pay with GC pauses (small but real), reduced control over memory layout, and a type system that is expressive enough for most things and frustrating for the rest.

Rust chose *control and a rich type system*. You get no runtime, no GC, predictable memory behaviour, and a type system that lets you encode a great deal in types. You pay with a steeper learning curve and code that is denser than Go's equivalent.

Pick Go when you are writing a service that is mostly I/O, where GC pauses are acceptable, and where team velocity matters more than per-request latency. Pick Rust when you care about tail latency, memory footprint, or safety guarantees that a GC cannot give you — or when you are writing code that has no GC to begin with, like an embedded device, a browser engine, or a kernel.

1.6 What you give up

Honesty is the best marketing. Rust has real costs, and the book would be dishonest if we did not name them.

- **Compile times.** Rust compiles slowly compared to Go, especially on cold builds. Incremental builds are faster; the compiler has gotten dramatically better over the years; but if fifteen-second edit-compile-test cycles matter to you, Rust will feel slower than what you are used to.
- **The learning curve.** Chapter 4 is the reason a lot of people bounce off Rust. Ownership is genuinely new, and the compiler will reject code that looks obviously correct to a programmer used to a GC'd language. This is not a flaw; it is the mechanism by which Rust's guarantees exist. It is also, however, real.
- **Boilerplate in certain patterns.** Heterogeneous collections, graphs with cycles, self-referential structures, and some async patterns all require more ceremony in Rust than in a GC'd language. Sometimes the ceremony is the point (the compiler is pushing you toward a cleaner design); sometimes it is friction you just pay.
- **Ecosystem maturity in narrow domains.** The mainstream is strong: web services, CLI tools, parsers, networking, cryptography, embedded. Niche domains — scientific computing with established Fortran libraries, deep learning at the bleeding edge, GUI frameworks for desktop apps — are catching up but are not yet at parity with Python or C++.

If none of these are disqualifying for your use case, keep reading.

1.7 A look ahead

In the next chapter we install Rust and meet `cargo`, the build tool and package manager that makes working in the language so pleasant that people complain when they have to use anything else. After that, Chapter 3 covers the pieces of the language that you would recognise from any curly-brace language — values, types, control flow — and from Chapter

4 onward we are in the part of the language that only Rust has.

Next: Chapter 2 — Environment

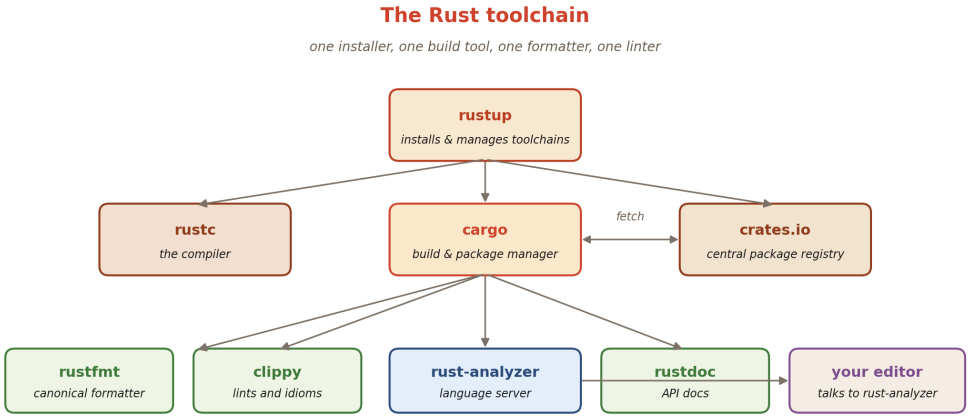
CHAPTER

Environment

Rust's tooling is not a side detail of the language — it is one of the reasons the language feels the way it does. You install *one* thing, and from it flows a compiler, a package manager, a formatter, a linter, a documentation generator, and a language-server that powers autocompletion in every editor people actually use. The tools do not argue with each other. They all agree on what a Rust project looks like. The result is that setting up a new project, adding a dependency, running tests, generating docs, and publishing to the world are all `cargo` commands with obvious names.

This chapter is a guided tour of those tools. By the end of it you will have Rust installed, a working project, and a feel for the day-to-day loop that Rust programmers live in.

2.1 The toolchain



Rustup manages toolchains. Cargo drives builds and fetches from crates.io. Editors talk to rust-analyzer.

The picture you want in your head is this:

- **rustup** is the installer and toolchain manager. It fetches and upgrades Rust itself; it lets you have multiple versions side by side (stable, beta, nightly, an older pinned version for a specific project); it installs compilation targets (so you can cross-compile for a Raspberry Pi from your laptop).
- **rustc** is the compiler. You will rarely invoke it directly; `cargo` calls it for you.
- **cargo** is the build system and package manager. It knows how to turn a directory with a `Cargo.toml` into a binary, a library, a test run, a documentation site, or a publishable release. It fetches dependencies from **crates.io**, the central registry of open-source Rust packages.
- **rustfmt** formats your code. There is exactly one canonical style; arguments about tabs-versus-spaces do not happen in Rust code reviews, which is a small daily gift.
- **clippy** is the linter. It catches idiom violations, common bugs, and stylistic mistakes. It is strict by default and almost always right.
- **rust-analyzer** is the language server. It powers completion, hover, inline errors, and refactorings inside your editor. It speaks the Language Server Protocol, so it works in VS Code, Zed, Neovim, Helix, JetBrains IDEs, Emacs — everywhere.

2.2 Installing Rust

The canonical installer is `rustup`. On macOS and Linux:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

On Windows, download `rustup-init.exe` from *rustup.rs* and run it. Either way, the installer asks a couple of questions (defaults are fine), then writes `cargo` and `rustc` into `~/.cargo/bin` and adds that directory to your `PATH`.

Restart your shell — or `source $HOME/.cargo/env` — and you should have:

```
$ rustc --version
rustc 1.82.0 (...)
$ cargo --version
cargo 1.82.0 (...)
```

To upgrade the toolchain later:

```
rustup update
```

Tip

If your workplace requires a specific Rust version, put a `rust-toolchain.toml` file at the root of the project:

```
[toolchain]
channel = "1.82.0"
components = ["rustfmt", "clippy"]
```

`rustup` will automatically install and use that version inside the project, even if your default is different.

2.3 Hello, cargo

Create a new binary project:

```
$ cargo new hello
     Created binary (application) `hello` package
$ cd hello
$ tree .
.
■■■ Cargo.toml
■■■ .gitignore
■■■ src
    ■■■ main.rs
```

`Cargo.toml` is the package manifest. A fresh one looks like this:

```toml
[package]
name = "hello"
version = "0.1.0"
edition = "2021"

[dependencies]
```

The `edition` field pins the *language edition*. Editions let Rust add features and minor breaking changes without invalidating old code — a 2018-edition crate can depend on a 2021-edition crate and vice versa. Use the newest edition for new projects.

`src/main.rs` contains the obligatory first program:

```rust
fn main() {
    println!("Hello, world!");
}
```

Now the verbs. These are the ones you will type a hundred times a day:

```
cargo build        # compile (debug mode, fast, no optimisations)
cargo run          # compile + run
cargo build --release   # compile optimised (slow to build, fast to run)
cargo test         # run all tests
cargo check        # type-check without generating a binary (fast)
cargo doc --open   # build HTML docs for your crate + deps, open in browser
cargo fmt          # format every file in the project
cargo clippy       # run the linter
```

`cargo check` is the one you will use most while developing. It skips code generation, so it is dramatically faster than `cargo build`, and rust-analyzer runs it in the background to

keep your editor's errors up to date.

2.4 Adding a dependency

Pick a crate from crates.io. Let us add the `rand` crate to our hello project:

```
$ cargo add rand
    Updating crates.io index
      Adding rand v0.8.5 to dependencies
```

`cargo add` edits `Cargo.toml` for you:

```
[dependencies]
rand = "0.8.5"
```

The version string `"0.8.5"` is a *semver caret constraint*: it means "anything compatible with 0.8.5". For 0.x versions (`major = 0`), that means the same `0.8.*` line; for 1.x and later, it means the same major version. `cargo` records the exact resolved versions in `Cargo.lock`, which you should commit for application crates and (usually) not commit for library crates.

Now use it:

```
use rand::Rng;

fn main() {
    let mut rng = rand::thread_rng();
    let n: u32 = rng.gen_range(1..=100);
    println!("random number in [1, 100]: {n}");
}
```

`cargo run` downloads, compiles, caches, and runs. The download happens once per version per machine; subsequent builds are from the local cache.

2.5 Tests

Rust ships testing in the standard library. A test is a function annotated with `#[test]`. Put them at the bottom of any `.rs` file, or in a separate `tests/` directory for integration tests.

Edit `src/main.rs`:

```
fn double(x: i32) -> i32 {
    x * 2
}

fn main() {
    println!("{}", double(21));
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn doubles_positive() {
        assert_eq!(double(3), 6);
    }

    #[test]
    fn doubles_negative() {
        assert_eq!(double(-4), -8);
    }

    #[test]
    fn doubles_zero() {
        assert_eq!(double(0), 0);
    }
}
```

Run them:

```
$ cargo test
   Compiling hello v0.1.0 (/.../hello)
    Finished test [unoptimized + debuginfo] target(s) in 0.52s
     Running unittests src/main.rs (target/debug/deps/hello-...)

running 3 tests
test tests::doubles_zero ... ok
test tests::doubles_negative ... ok
test tests::doubles_positive ... ok

test result: ok. 3 passed; 0 failed; 0 ignored
```

The `#[cfg(test)]` attribute tells the compiler to include that module only when building tests. `use super::*;` pulls in the parent module's items so the tests can see `double`.

2.6 Formatting and linting

Run these before you commit:

```
cargo fmt           # formats in place
cargo fmt --check   # fails CI if anything would change (no-op mode)
cargo clippy        # lints
cargo clippy -- -D warnings   # treat lint warnings as errors
```

`rustfmt` has one canonical style and a small set of tunables. The project convention is *use the default*; bikeshedding over braces has been outsourced to a tool that does not care.

`clippy` is the real-world gem. It knows the idioms, flags the anti-patterns, and suggests fixes. A typical warning:

```
warning: this `.clone()` is redundant
  --> src/main.rs:4:15
   |
 4 |     let y = x.clone();
   |               ^^^^^^^^ help: remove this
   |
   = note: `#[warn(clippy::redundant_clone)]` on by default
```

The first time you run `clippy` on someone else's code, set aside an afternoon. The first time you run it on your own, set aside an hour and accept that you will be a better Rust programmer after.

2.7 Editors

`rust-analyzer` is the canonical language server. Install the plugin for your editor of choice:

- **VS Code** — `rust-lang.rust-analyzer` from the marketplace.
- **Neovim / Helix / Zed** — built in or one-line config.
- **JetBrains** — the *RustRover* IDE, or the Rust plugin for IDEA.

Open a cargo project in your editor, and the language server will:

- Highlight errors as you type.
- Show inferred types as inlay hints.
- Provide completion, hover documentation, go-to-definition.
- Offer refactorings (extract function, inline variable, add missing trait impls).
- Run tests with a click on the `#[test]` attribute.

It is, without exaggeration, the best language server for any language I have used. Treat it as mandatory, not optional.

2.8 The workspace

A *workspace* is one `Cargo.toml` that ties several crates together. For a single-binary hobby project you will not need one; for anything with more than two crates, you will.

A workspace manifest at the root:

```
# Cargo.toml
[workspace]
members = ["app", "core", "cli"]
resolver = "2"

[workspace.dependencies]
serde = { version = "1", features = ["derive"] }
tokio = { version = "1", features = ["full"] }
```

Each member crate keeps its own `Cargo.toml` with a slim `[dependencies]` section that pulls versions from the workspace:

```
# app/Cargo.toml
[package]
name = "app"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { workspace = true }
core = { path = "../core" }
```

Running `cargo build` at the workspace root builds every member. This is how most real Rust applications are organised.

2.9 A 30-second loop

Here is the actual rhythm of working in Rust, once the tools are installed:

1. Write code in your editor, with rust-analyzer showing you types and errors as you type.
2. Save. rust-analyzer runs `cargo check` in the background and updates the error pane.
3. When the file is clean, run `cargo test` in a terminal.
4. When a feature is done, run `cargo fmt` and `cargo clippy`.
5. When a milestone is done, run `cargo build --release` and benchmark if performance matters.

No IDE configuration, no project-generator plugins, no fights with the build tool. Cargo is boring in the best way. The next chapter is where Rust stops looking like every other language and starts looking like itself.

Next: Chapter 3 — Core Language

CHAPTER

Core Language

Before we meet the parts of Rust that no other language has, we need to cover the parts that every language has: values, types, control flow, functions. You will recognise most of this chapter from any curly-brace language you already know — but even in the familiar bits, Rust has small, sharp differences that matter later. `let` *bindings are immutable by default.* `if` *is an expression. Integer overflow panics in debug and wraps in release.* Skim for the differences; do not skim for the basics.

3.1 Variables

A binding is introduced with `let`:

```
let x = 5;
let name = "Ada";
```

`x` and `name` cannot be reassigned. To make a binding mutable, say so with `mut`:

```
let mut counter = 0;
counter += 1;
counter += 1;
println!("{counter}");    // 2
```

Immutability is the default because most variables *are* constants once initialised, and the `mut` keyword is a tiny flag that tells a reader "this will change". You will soon find yourself annoyed when you return to a language where every binding is mutable.

Shadowing is different from mutation. A fresh `let` with the same name creates a *new* binding that hides the old one:

```
let x = 5;
let x = x + 1;          // x is now a new binding, still immutable
let x = "hello";        // the type can even change
```

This is not mutation — each `x` is a distinct variable. It is a common idiom for the "parse-then-use" pattern:

```
let input = "42";               // &str
let input: i32 = input.parse().unwrap();   // shadow with i32
```

3.2 Primitive types

Rust is statically typed. Every value has a type known at compile time. The primitive numeric types are:

Family

Sizes

Notes

Signed

`i8`, `i16`, `i32`, `i64`, `i128`, `isize`

two's complement

Unsigned

`u8` , `u16` , `u32` , `u64` , `u128` , `usize`

Float

`f32` , `f64`

IEEE 754

Bool

`bool`

`true` / `false`

Char

`char`

4 bytes, Unicode scalar

`isize` and `usize` are pointer-sized — 64-bit on a 64-bit machine, 32-bit on a 32-bit one. Array indexing uses `usize` .

Integer literals may have a type suffix:

```
let pixel: u8   = 255;
let big_number  = 1_000_000_u64;   // underscores for readability
let hex         = 0xCAFE;
let octal       = 0o77;
let binary      = 0b1010_0110;
```

Without a suffix or annotation, an integer literal defaults to `i32` , and a float to `f64` .

> **Warning**
>
> **Integer overflow.** In debug builds, overflow *panics*. In release builds, it *wraps*. This is deliberate: debug catches bugs, release runs fast. If you need defined semantics in both modes, use the explicit methods: `wrapping_add` , `checked_add` , `saturating_add` , or `overflowing_add` .

3.3 Compound types

Tuples group a fixed number of values of (possibly) different types:

```
let pair: (i32, f64) = (1, 2.5);
let (a, b) = pair;              // destructure
println!("{} {}", pair.0, pair.1);  // or index with .N
```

The empty tuple `()` is called the *unit*. It is the type of expressions that do not produce a meaningful value — which in a curly-brace language is `void`, but in Rust is a real type that can be assigned, matched on, returned, and passed around.

Arrays are fixed-length sequences of the *same* type, stored inline (on the stack by default):

```
let xs: [i32; 4] = [1, 2, 3, 4];
let zeros = [0; 16];            // 16 zeros: [0, 0, ..., 0]

println!("{}", xs[2]);           // 3
println!("{}", xs.len());        // 4
```

The length is part of the type. `[i32; 4]` and `[i32; 5]` are different types. For a growable, heap-allocated sequence — the thing you usually want — use `Vec<T>` (Chapter 7).

Slices are views into arrays or vectors. They are a *pair of pointer and length*, so they know how long they are, but they do not own the underlying storage:

```
let xs = [1, 2, 3, 4, 5];
let middle: &[i32] = &xs[1..4];   // &[2, 3, 4]
```

A string slice `&str` is just a slice of UTF-8 bytes you are guaranteed is valid UTF-8. An owned, growable string is `String`. The pair (`&str`, `String`) parallels (`&[T]`, `Vec<T>`):

```
let literal: &str = "hello";        // lives in the binary
let owned: String = String::from("hello");
let borrowed: &str = &owned;        // slice into the String
```

We will return to this in Chapter 4 — the reason there are two types is *ownership*, and it will make sense then.

3.4 Control flow

`if` looks normal and is almost normal:

```
let n = 7;
if n % 2 == 0 {
    println!("even");
} else if n % 3 == 0 {
    println!("divisible by three");
} else {
    println!("odd");
}
```

But `if` is an *expression*, not a statement. Every branch produces a value; all branches must have the same type:

```
let parity = if n % 2 == 0 { "even" } else { "odd" };
```

`loop`, `while`, and `for` are the three looping constructs:

```
// Infinite loop — useful when you break with a value.
let answer = loop {
    // ... some work ...
    if ready { break 42; }
};

// Conditional loop.
let mut i = 0;
while i < 5 {
    print!("{i} ");
    i += 1;
}

// For-each loop — by far the most common.
for x in [10, 20, 30] {
    print!("{x} ");
}

for i in 0..10 { /* 0, 1, ..., 9 */ }
for i in 0..=9 { /* same, inclusive form */ }
```

`for` is the idiomatic loop. You will rarely write indices by hand; when you do need an index, use `.enumerate()`:

```
for (i, name) in ["Ada", "Grace", "Linus"].iter().enumerate() {
    println!("{i}: {name}");
}
```

`match` is the most powerful control-flow construct in the language. It dispatches on the *shape* of a value:

```
let n = 3;
let label = match n {
    0 => "zero",
    1 | 2 => "small",
    3..=9 => "medium",
    _ => "big",
};
println!("{label}");   // medium
```

`match` is *exhaustive*: you must handle every possible value, or the compiler will tell you what is missing. This is what makes it safe to dispatch on the variants of an `enum` (Chapter 5) — you cannot accidentally forget a case.

3.5 Functions

A function declaration looks like this:

```
fn add(x: i32, y: i32) -> i32 {
    x + y
}
```

Three things to notice:

1. **Parameter types are mandatory.** Rust does not infer them. (It infers *local* types aggressively, though.)
2. **The return type follows `->`.** If you omit it, the function returns `()`, the unit type.
3. **The last expression is the return value.** There is no semicolon after `x + y`. If you write `x + y;` (with a semicolon), the expression becomes a *statement* that evaluates to `()`, and you will get a type error because `()` is not `i32`.

You *can* use `return` explicitly for early exit:

```rust
fn clamp_positive(x: i32) -> i32 {
    if x < 0 {
        return 0;
    }
    x
}
```

But the idiomatic form at the tail of a function is the expression-without-semicolon.

> **Note**
>
> **Statements versus expressions.** In Rust almost everything is an expression: `if`, `match`, blocks (`{ … }`), `loop`, even assignments (which evaluate to `()`). Statements are terminated by `;`; an expression without `;` produces a value. This distinction is what lets `let y = if cond { 1 } else { 2 };` be a legal binding.

Multiple return values use a tuple:

```rust
fn min_max(xs: &[i32]) -> (i32, i32) {
    let mut lo = xs[0];
    let mut hi = xs[0];
    for &x in xs {
        if x < lo { lo = x; }
        if x > hi { hi = x; }
    }
    (lo, hi)
}

let (a, b) = min_max(&[3, 1, 4, 1, 5, 9, 2, 6]);
```

For more than two or three return values, give them names by returning a struct (Chapter 5) — tuples get unreadable fast.

3.6 Blocks are expressions

A block `{ … }` is itself an expression. Its value is the value of its last expression (or `()` if the block ends in a statement).

```
let area = {
    let w = 10;
    let h = 20;
    w * h            // no semicolon — this is the block's value
};
assert_eq!(area, 200);
```

This is not a stylistic curiosity. It lets you scope temporary variables cleanly (`w` and `h` do not leak out of the block), and it combines with `if` / `match` / `loop` to form the functional expressions that Rust programs tend to be made of.

3.7 Printing and formatting

The two workhorses are `println!` and `format!`:

```
let name = "Ada";
let age = 36;

println!("{name} is {age}");                    // Ada is 36
println!("{} is {}", name, age);                // same, positional
println!("{age:5}");                            // right-pad to width 5
println!("{age:05}");                           // pad with zeros
println!("{pi:.3}", pi = std::f64::consts::PI);// three decimal places
println!("{n:08b}", n = 42u8);                  // binary, 8 wide, zero-padded
println!("{:?}", [1, 2, 3]);                    // debug formatting
println!("{:#?}", [1, 2, 3]);                   // pretty-printed debug
```

Two format traits are worth naming early:

- `Display` is for user-facing output (`{}`). You implement it manually for your own types.
- `Debug` is for programmer-facing output (`{:?}`). You usually derive it automatically with `#[derive(Debug)]` on structs and enums.

We will meet both again in Chapter 5.

3.8 Modules and visibility

A module is a namespace. Inside a file, you create one with `mod`:

```
mod math {
    pub fn square(x: i32) -> i32 { x * x }
    fn private_helper() {}      // not visible outside `math`
}

fn main() {
    let y = math::square(5);
    println!("{y}");
}
```

Items are private by default. `pub` makes them visible to the parent module. For larger projects, split modules into files:

```
src/
■■■ main.rs
■■■ math.rs             ← mod math; in main.rs
■■■ math/
    ■■■ linalg.rs       ← mod linalg; in math.rs
```

`use` brings names into scope:

```
use std::collections::HashMap;
use std::io::{self, Read, Write};   // groups
```

3.9 Standard output and input

`println!` writes to standard output with a newline; `eprintln!` writes to standard error. Reading a line from standard input:

```
use std::io::{self, BufRead};

fn main() {
    let stdin = io::stdin();
    let line = stdin.lock().lines().next().unwrap().unwrap();
    println!("you said: {line}");
}
```

The double `unwrap()` is a small mystery that Chapter 6 will resolve: the first one unwraps an `Option` (was there a next line?), the second unwraps a `Result` (did reading succeed?). For now, take it on faith.

3.10 A full small program

Putting it together, here is a program that reads integers from stdin and prints their histogram:

```
use std::io::{self, BufRead};

fn main() {
    let stdin = io::stdin();
    let mut counts = [0u32; 10];          // counts[d] = number of digits == d

    for line in stdin.lock().lines() {
        let line = line.unwrap();
        for c in line.chars() {
            if let Some(d) = c.to_digit(10) {
                counts[d as usize] += 1;
            }
        }
    }

    for (d, &count) in counts.iter().enumerate() {
        let bar: String = "#".repeat(count as usize);
        println!("{d}: {bar} ({count})");
    }
}
```

Everything in this program uses ideas from this chapter. `let mut`, `for` loop, iterator methods, tuple destructuring with `enumerate`, formatted printing, the `if let` pattern (a one-armed match, which Chapter 5 formalises).

In the next chapter we meet the idea that is unlike anything in other languages: the rules the Rust compiler enforces to decide *who owns* the values your program creates, and therefore *who must free them*.

Next: Chapter 4 — Ownership, Borrowing, Lifetimes

CHAPTER

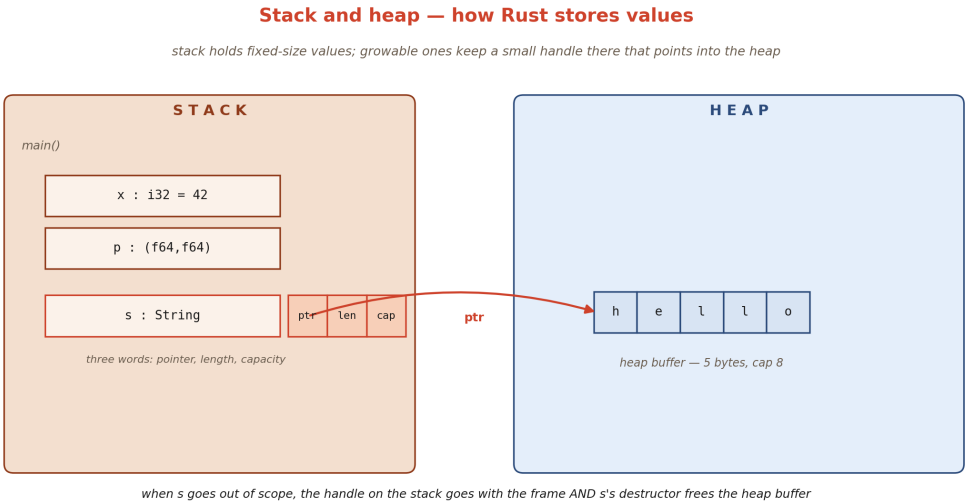
Ownership, Borrowing, Lifetimes

This is the chapter. Every promise the language makes — no use- after-free, no double-free, no data race, no null dereference — is enforced by the rules you meet here. These rules are not a runtime check. They are not a best practice. They are the meaning of a valid Rust program. If your code compiles, it obeys them; if your code does not compile, it is because the compiler can prove it does not obey them.

You will fight these rules. Every new Rust programmer does. The fight is short, because the rules are small in number and entirely consistent; but the fight is real, because the rules sometimes reject code that looks obviously correct, and you have to learn to see the hidden structure that makes it *not* correct. The dividend, when you win, is a kind of programming that does not leak memory and does not segfault and does not race, and that is worth the dust-up.

We will cover four topics, in order: **ownership**, **move semantics**, **borrowing**, **lifetimes**.

4.1 Stack and heap



Values live on the stack by default. `Box`, `String`, `Vec` store a small handle on the stack that points into the heap.

A quick refresher before we start, because Rust will be more explicit about this than your last language was.

The stack is a region of memory that grows and shrinks as functions are called and return. Every local variable — every `let` binding — lives on the stack of the function that declared it. Stack memory is automatically reclaimed when the function returns. Stack allocations are cheap: one arithmetic operation on the stack pointer.

The heap is a region of memory managed by an allocator. When you ask for heap memory (via `String`, `Vec`, `Box`, etc.), the allocator finds you a chunk and hands back a pointer. Heap allocations are comparatively expensive, and the memory must be *freed* explicitly to avoid leaks.

Values of known, fixed size — integers, bools, floats, characters, tuples of such values, arrays of such values — live on the stack. Values that can grow (like a `String` that reads a file) store their variable-size payload on the heap; the stack holds a small, fixed- size *handle* that contains (pointer, length, capacity).

Knowing which is which is not about performance (though it matters for that too). It is about understanding what Rust needs to track. The stack manages itself; the heap needs someone to decide when the memory is no longer needed. In C, that someone is you. In Go, that someone is the garbage collector. In Rust, that someone is *the ownership system*.

4.2 Ownership

Every value has exactly one owner at any moment in time. When the owner goes out of scope, the value is *dropped* — its destructor runs and its resources (including any heap memory) are freed.

```
fn main() {
    let s = String::from("hello");    // s owns the heap buffer for "hello"
    println!("{s}");
}                                      // s goes out of scope; "hello" is freed
```

That is the whole idea. Nothing else in this chapter is more than a consequence of it. Two questions immediately arise: what happens when you try to *transfer* ownership, and what happens when you try to *share* a value without transferring it? The answers are **move semantics** and **borrowing**.

4.3 Move semantics

Assigning a heap-backed value to a new variable *moves* it.

```
let s1 = String::from("hello");
let s2 = s1;                // s1's ownership is moved to s2
// println!("{s1}");        // ERROR: borrow of moved value
println!("{s2}");           // OK
```

After `let s2 = s1;`, the name `s1` is no longer usable. The heap buffer still exists, but it is now reachable only through `s2`. When `s2` goes out of scope, the buffer is freed exactly once.

This is how Rust avoids double-free without a garbage collector: ownership is *linear*, and the compiler enforces it.

```
error[E0382]: borrow of moved value: `s1`
  --> src/main.rs:4:20
   |
 2 |     let s1 = String::from("hello");
   |         -- move occurs because `s1` has type `String`
 3 |     let s2 = s1;
   |              -- value moved here
 4 |     println!("{s1}");
   |                ^^^^ value borrowed here after move
```

Read that message carefully. Every new Rust programmer sees it a hundred times. The fix depends on what you actually meant:

- If you meant "give me a second copy of the string", call `s1.clone()`. Cloning a `String` allocates a new heap buffer and copies the bytes. It is not free.
- If you meant "let me look at the string without taking it", take a *reference*: `let r = &s1;` (see §4.5).
- If you meant "hand ownership to s2", then you already did; stop using s1 after the move.

The Copy trait

Not every type is moved on assignment. Primitive types — integers, floats, booleans, characters, and tuples/arrays made entirely of them — implement a marker trait called `Copy`. For `Copy` types, assignment *copies* the value bit-for-bit, and both names remain valid:

```
let x = 5;
let y = x;
println!("{x} {y}");   // both valid — i32 is Copy
```

`Copy` only applies to types that are trivially duplicable by copying their bits and do not own anything on the heap. `String` is *not* `Copy` (because two `String`s would both try to free the same buffer). `Vec<T>` is *not* `Copy`. `Box<T>` is *not* `Copy`. You cannot derive `Copy` on a type that contains a non-`Copy` field.

> **Note**
> **Clone versus Copy.** `Clone` is the general "make a new copy" operation; it is *explicit* (you call `.clone()`) and may allocate. `Copy` is a *promise* that cloning is bitwise and always free, and it is automatic on assignment. Every `Copy` type is also `Clone`; the reverse is not true.

Moves in functions

Passing a value to a function is a move, for the same reasons:

```
fn take(s: String) {
    println!("{s}");
}                           // s dropped here

fn main() {
    let greeting = String::from("hello");
    take(greeting);
    // println!("{greeting}");   // ERROR: moved into take()
}
```

If you want to keep using the value after the call, don't give it away. Pass a *reference* (§4.5), or have the function give it back:

```
fn use_and_return(s: String) -> String {
    println!("{s}");
    s                           // return it, transferring ownership back
}
```

Returning ownership works, but it is clumsy for more than one value. Borrowing is the clean way.

4.4 Scope and drop

Before we introduce borrowing, one more thing about ownership: *when* is a value dropped?

```
fn main() {
    let a = String::from("outer");
    {
        let b = String::from("inner");
        println!("{a} {b}");
    }                               // b dropped here
    println!("{a}");
}                                   // a dropped here
```

Drops happen in *reverse order of declaration* within a scope. This is the same rule C++ uses for RAII. It matters for types that do real work in their destructor — file handles, database connections, mutex guards — because it means you can predict exactly when those resources are released.

You can drop early with `std::mem::drop`:

```
let v = vec![1, 2, 3];
// ... use v ...
drop(v);                        // free now, not at end of scope
```

But you almost never need to. Rust's RAII is already tight enough.

4.5 Borrowing

You usually want to *use* a value without taking ownership of it. A reference lets you do that:

```
fn len(s: &String) -> usize {       // s is a reference to a String
    s.len()
}                                   // s goes out of scope, but
                                    // it never owned the string

fn main() {
    let greeting = String::from("hello");
    let n = len(&greeting);         // pass a reference
    println!("{greeting} has {n} bytes");  // greeting still valid
}
```

`&T` is a *shared reference* (also called an *immutable reference* or a *borrow*). A shared reference lets you read a value but not modify it. You can have as many shared references as you like, all active at the same time.

To *modify* through a reference, use `&mut T`, a *mutable reference*:

```rust
fn push_dot(s: &mut String) {
    s.push('.');
}

fn main() {
    let mut greeting = String::from("hello");
    push_dot(&mut greeting);
    println!("{greeting}");        // hello.
}
```

Two things to notice:

1. `greeting` must be declared `mut`. You cannot take `&mut` of an immutable binding.
2. At the call site you write `&mut greeting` — the `mut` is not implicit. You *always* see in the source code when a reference might mutate.

The borrowing rules

These are the two rules that the compiler enforces at every program point:

1. **At any given time, you may have either one mutable reference or any number of shared references — but not both.**
2. **References must always be valid** (not dangling).

Rule 1 prevents data races *within a single thread* — and, via the `Send` / `Sync` traits (Chapter 8), across threads too. Two pointers to the same memory cannot both write, and if either one writes, no one else may even read.

Rule 2 prevents use-after-free. A reference cannot outlive the data it points to.

Here is what rule 1 looks like in practice:

```rust
let mut v = vec![1, 2, 3];

let r1 = &v;                    // shared borrow #1
let r2 = &v;                    // shared borrow #2 — fine
println!("{:?} {:?}", r1, r2);

let m = &mut v;                 // ERROR while r1/r2 still live
```

The fix is to let the shared borrows end before the mutable one begins. Happily, the compiler does this automatically now via *non-lexical lifetimes*: a reference's lifetime is the span from its creation to its *last use*, not the end of its enclosing block. So this works:

```
let mut v = vec![1, 2, 3];
let r1 = &v;
let r2 = &v;
println!("{:?} {:?}", r1, r2);  // r1 and r2 last used here
let m = &mut v;                 // OK — r1/r2 are no longer live
m.push(4);
```

Why the rules exist

Consider what would happen without rule 1. Here is a bug in C that every systems programmer has written at least once:

```
/* C — compiles and runs, may crash or corrupt memory */
char *s = malloc(4);
strcpy(s, "hi!");
char *t = s;
free(s);
printf("%s\n", t);   /* use-after-free */
```

And another:

```
/* C — iterator invalidation */
for (int i = 0; i < vec->len; i++) {
    if (should_add) vec_push(vec, vec->data[i]);  /* oops */
}
```

The first bug violates rule 2. The second violates rule 1 — `vec->data[i]` is a read, `vec_push` may be a write, and if the push reallocates the backing buffer, `vec->data[i]` is now a dangling pointer. Rust rejects both patterns at compile time. You cannot write the first because `t` would be a reference to data that has been freed. You cannot write the second because inside the loop you are iterating over `&vec`, and a call that takes `&mut vec` cannot coexist with it.

Dereferencing and automatic derefencing

`*r` dereferences a reference explicitly:

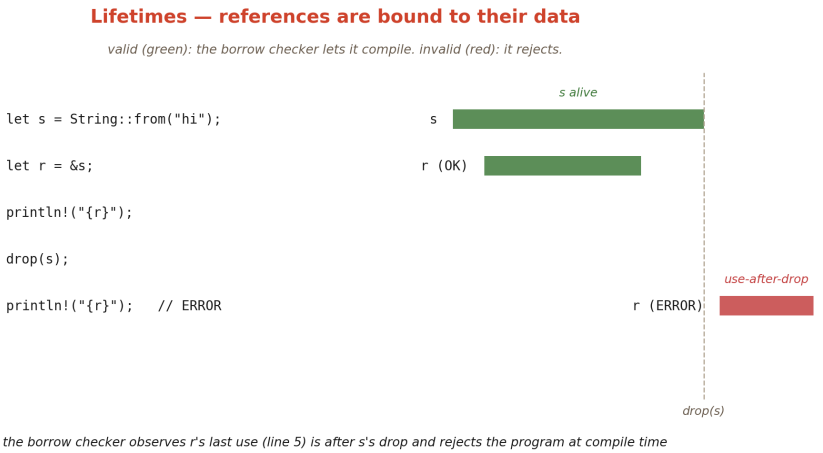
```
let x = 5;
let r = &x;
assert_eq!(*r, 5);
```

But you rarely need to write `*`. When you call a method, Rust auto-dereferences for you:

```
let s = String::from("hello");
let r = &s;
println!("{}", r.len());        // same as (*r).len()
```

And when you pass a `&String` where `&str` is expected, Rust auto-coerces (via the `Deref` trait). This is called *deref coercion* and is the reason you can almost always forget about the distinction in practice.

4.6 Lifetimes



Every reference has a lifetime — the scope during which it is valid. The borrow checker rejects references that outlive their data.

A lifetime is the span of code during which a reference is valid. Most of the time the compiler infers lifetimes for you, and you never see them. But sometimes — in function signatures, in struct definitions — you have to write them down, and understanding what they mean is the last step in learning the borrow checker.

The problem lifetimes solve

Consider this function:

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}
```

This will not compile. The compiler sees that the returned reference points into either `x` *or* `y`, and it cannot tell which — and therefore cannot tell how long the returned reference is valid. It asks us to say:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

`'a` is a *lifetime parameter*. Read the signature as: "for any lifetime `'a`, if you give me two `&str`s that both live at least as long as `'a`, I will give you back a `&str` that also lives as long as `'a`." In practice, `'a` will be the *shorter* of the two input lifetimes — which is exactly what we want, because the returned reference might point into either.

When you must write lifetimes

Most of the time, you don't. The compiler applies a set of *elision* rules to fill them in for you:

1. Every input reference gets its own lifetime parameter.
2. If there is exactly one input lifetime, it is assigned to all output references.
3. If one of the inputs is `&self` or `&mut self`, its lifetime is assigned to all output references.

So this function:

```
fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}
```

…has a single input reference, so rule 2 applies: the output has the same lifetime as `s`. You do not write `'a` anywhere, and the compiler understands.

You need to write lifetimes when:

- A function takes multiple references and returns one, and the compiler cannot tell which input the output points into.
- A struct contains a reference.
- You want to constrain a generic type parameter to outlive something.

References in structs

A struct that holds a reference must name its lifetime:

```
struct Excerpt<'a> {
    text: &'a str,
}

impl<'a> Excerpt<'a> {
    fn new(text: &'a str) -> Self {
        Excerpt { text }
    }

    fn text(&self) -> &str {
        self.text
    }
}

fn main() {
    let novel = String::from("It was the best of times.");
    let e = Excerpt::new(&novel);
    println!("{}", e.text());
}                                   // novel outlives e — fine
```

The lifetime parameter `'a` is a *constraint*: an `Excerpt<'a>` cannot outlive whatever `'a` refers to. If you tried to make the `Excerpt` live longer than `novel`, the compiler would reject it.

The `'static` lifetime

`'static` is the lifetime of values that live for the entire program:

```
let s: &'static str = "hello world";   // lives in the binary
```

String literals are `&'static str` because they are baked into the executable. You will see `'static` as a bound on some APIs (for instance, values passed to

`std::thread::spawn` must be `'static` because the thread may outlive the spawning function).

4.7 A worked example

The canonical exercise is to write a function that takes a string and returns the first word in it:

```
fn first_word(s: &str) -> &str {
    let bytes = s.as_bytes();
    for (i, &b) in bytes.iter().enumerate() {
        if b == b' ' {
            return &s[..i];
        }
    }
    s
}

fn main() {
    let sentence = String::from("hello world");
    let w = first_word(&sentence);
    println!("first word: {w}");
}
```

The return type is `&str`. Why not `String`? Because we do not need a new allocation — we just want to point into the string we were given. The lifetime elision rules give the output the same lifetime as the input, so the compiler knows `w` is valid as long as `sentence` is.

If you added this line right before the `println!`:

```
sentence.clear();
println!("first word: {w}");
```

…you would get a compile error. `sentence.clear()` takes `&mut sentence`, but `w` is still alive and holds a `&` into it. The borrow checker rejects the code *before* it can crash. This is what "if it compiles, it does not segfault" means.

4.8 Reading compiler messages

Rust's compiler is uncommonly good at explaining itself. When the borrow checker objects, read the whole message:

```
error[E0502]: cannot borrow `v` as mutable because it is also
              borrowed as immutable
  --> src/main.rs:5:5
   |
 3 |     let r = &v;
   |             - immutable borrow occurs here
 4 |     v.push(4);
   |     ^^^^^^^^^ mutable borrow occurs here
 5 |     println!("{:?}", r);
   |                      - immutable borrow later used here
```

The message tells you:

- *what* the rule violation is (simultaneous mutable + immutable),
- *where* the competing borrows live (lines 3 and 4),
- *why* it matters (the immutable borrow is used again on line 5).

Once you train your eye to this format, the compiler becomes a teacher. Nine times in ten, the fix is obvious once you see all three pieces at once.

4.9 Summary

The ownership rules, one more time, with nothing around them:

- Every value has exactly one owner.
- Assigning or passing a non-`Copy` value *moves* it.
- A value is *dropped* when its owner goes out of scope.
- You may have any number of `&T` references, **or** exactly one `&mut T` reference, at a time.
- A reference must never outlive the data it points to.

Everything else in Rust — traits, generics, error handling, concurrency — is built on these five sentences. The next chapter starts building.

Next: Chapter 5 — Types: Structs, Enums, Traits, Generics

CHAPTER

Types: Structs, Enums, Traits, Generics

Rust's type system is where the language pays its rent. The ownership rules you learned in Chapter 4 are enforced through types; the zero-cost abstractions the language promises are built out of traits and generics; the pattern-matching that makes Rust feel so tight is powered by enums. Learning to *think in types* is the step from writing Rust that compiles to writing Rust that is good.

This chapter covers the four tools you compose to build every abstraction in the language: **structs**, **enums**, **traits**, and **generics**.

5.1 Structs

A `struct` is a named bundle of fields:

```
struct Point {
    x: f64,
    y: f64,
}

fn main() {
    let p = Point { x: 3.0, y: 4.0 };
    println!("({}, {})", p.x, p.y);
}
```

Fields are private to the module that defines the struct unless marked `pub`. Construction uses the name of every field — there are no positional arguments, which means a `Point` constructed today will be correct even if someone adds a third field tomorrow.

Tuple structs and unit structs

Two lightweight variants are available when names would add noise:

```rust
struct Meters(f64);            // tuple struct — a newtype wrapper
struct Marker;                 // unit struct — no data

let distance = Meters(1500.0);
println!("{}", distance.0);
```

Tuple structs are especially useful as *newtype wrappers* to stop accidental mixing of values with the same primitive representation:

```rust
struct Meters(f64);
struct Seconds(f64);

fn speed(m: Meters, s: Seconds) -> f64 {
    m.0 / s.0
}

let _v = speed(Meters(100.0), Seconds(9.58));
// let _v = speed(Seconds(9.58), Meters(100.0));  // TYPE ERROR
```

The compiler will not let you pass a `Seconds` where a `Meters` is expected, even though both are just `f64` at runtime.

Methods and `impl` blocks

Methods are defined in an `impl` block attached to the type:

```rust
struct Point { x: f64, y: f64 }

impl Point {
    // Associated function (no self) — typically used as a constructor.
    fn new(x: f64, y: f64) -> Self {
        Point { x, y }
    }

    // Method that borrows self.
    fn magnitude(&self) -> f64 {
        (self.x * self.x + self.y * self.y).sqrt()
    }

    // Method that borrows self mutably.
    fn scale(&mut self, k: f64) {
        self.x *= k;
        self.y *= k;
    }

    // Method that takes ownership of self — consumes the Point.
    fn into_tuple(self) -> (f64, f64) {
        (self.x, self.y)
    }
}

fn main() {
    let mut p = Point::new(3.0, 4.0);
    println!("{}", p.magnitude());   // 5.0
    p.scale(2.0);
    let (x, y) = p.into_tuple();     // p is consumed here
    println!("({x}, {y})");
}
```

Three flavours of `self` — `&self`, `&mut self`, `self` — map exactly to the three ownership modes you met in Chapter 4. Pick the least-powerful one that does the job: `&self` when you only read, `&mut self` when you modify, `self` only when the method truly *consumes* the value (for instance, `into_iter`, which turns a collection into an iterator that owns it).

`Self` with a capital S is the type of the thing being `impl`'d. In a `impl Point` block, `Self` means `Point`. It saves typing and makes trait definitions (§5.4) work cleanly.

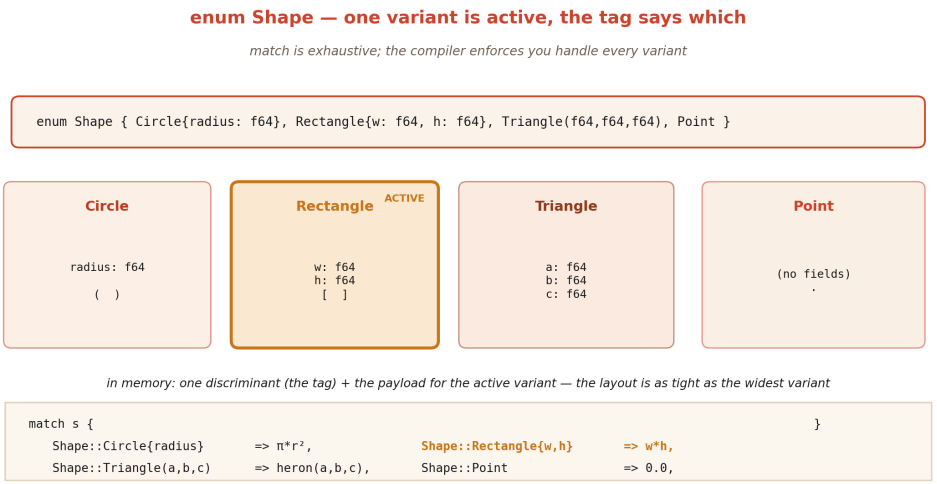
Struct update syntax

When you are making a small change to a struct, the update syntax saves typing:

```
let p1 = Point { x: 1.0, y: 2.0 };
let p2 = Point { x: 3.0, ..p1 };   // y from p1, x overridden
```

The `..p1` *moves* any fields not explicitly listed, and then `p1` is no longer usable (if any of its fields are non-`Copy`). For `Copy` types, `..p1` copies, and `p1` remains valid.

5.2 Enums



A Rust enum is a tagged union: one variant is active at a time, and the compiler forces you to handle each.

An `enum` is a type whose value is *exactly one* of a named set of variants. Each variant can carry its own data:

```
enum Shape {
    Circle { radius: f64 },
    Rectangle { width: f64, height: f64 },
    Triangle(f64, f64, f64),     // sides a, b, c
    Point,                        // no data
}
```

Construction names the variant:

```
let s = Shape::Circle { radius: 2.0 };
```

The key idea: an enum *unifies* distinct types under a single type. You can put circles, rectangles, and triangles into the same `Vec<Shape>` even though they do not have the same fields. The enum is a *tagged union* — one tag (which variant?) and one payload (the data for that variant) — and Rust lays it out efficiently.

Pattern matching

`match` dispatches on the variant and binds its fields:

```
fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle { radius } => std::f64::consts::PI * radius * radius,
        Shape::Rectangle { width, height } => width * height,
        Shape::Triangle(a, b, c) => {
            let s = (a + b + c) / 2.0;
            (s * (s - a) * (s - b) * (s - c)).sqrt()
        }
        Shape::Point => 0.0,
    }
}
```

`match` is *exhaustive*. Leave out a variant, and you get:

```
error[E0004]: non-exhaustive patterns: `&Shape::Point` not covered
```

This is the safety net. Add a new variant to `Shape` six months from now and the compiler will list every `match` in the codebase that needs updating. Enums plus exhaustive `match` are the single reason why extending a data model in Rust is pleasant while in most other languages it is terrifying.

`if let` and `while let`

When you only care about one variant, `if let` is a one-armed match:

```rust
let s = Shape::Circle { radius: 2.0 };

if let Shape::Circle { radius } = s {
    println!("a circle with r = {radius}");
} else {
    println!("not a circle");
}
```

`while let` loops as long as a pattern matches:

```rust
let mut stack = vec![1, 2, 3];
while let Some(top) = stack.pop() {
    println!("{top}");
}
```

Option and Result

The two most common enums in the standard library are so central you will meet them on page one of any Rust codebase:

```rust
enum Option<T> {
    Some(T),
    None,
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

`Option<T>` is Rust's *"might be absent"* type — the replacement for nullable pointers. `Result<T, E>` is Rust's *"might have failed"* type — the replacement for exceptions. Both get full treatment in Chapter 6.

Note that both are generic. `Option<i32>`, `Option<String>`, and `Option<Vec<User>>` are all different types. The `T` is a parameter, filled in by the caller, which brings us to…

5.3 Generics

A generic type or function takes a type parameter in angle brackets and then uses that parameter in its body:

```
fn largest<T: PartialOrd>(xs: &[T]) -> &T {
    let mut best = &xs[0];
    for x in xs {
        if x > best {
            best = x;
        }
    }
    best
}

fn main() {
    let ints = vec![3, 1, 4, 1, 5, 9, 2, 6];
    println!("{}", largest(&ints));        // 9

    let words = vec!["apple", "banana", "pear"];
    println!("{}", largest(&words));       // pear
}
```

`largest` works on any slice of `T`, so long as `T` implements `PartialOrd` (which defines `<`, `>`, etc.). The `T: PartialOrd` after the angle brackets is a *trait bound* — a promise that `T` supports the operation we need.

Generics are *monomorphised*: the compiler generates a specialised version of `largest` for each concrete `T` you call it with. There is no virtual dispatch, no boxing, no runtime type tag. Generic code is exactly as fast as hand-written specialised code. This is the mechanism behind "zero-cost abstraction".

Structs and enums can be generic too:

```
struct Pair<T> {
    first: T,
    second: T,
}

enum Either<L, R> {
    Left(L),
    Right(R),
}
```

Methods can add their own bounds:

```
impl<T: std::fmt::Display> Pair<T> {
    fn announce(&self) {
        println!("pair: {} and {}", self.first, self.second);
    }
}
```

Now `Pair<i32>` has an `announce` method (because `i32: Display`), but `Pair<Vec<u8>>` does not (because `Vec<u8>` does not implement `Display`). You get the method when the bound is satisfied, and you are told when it is not.

5.4 Traits

A `trait` is a set of methods that a type may choose to implement. It is Rust's answer to interfaces, type classes, and protocols.

```rust
trait Greet {
    fn hello(&self) -> String;

    // A default method — impls may override.
    fn shout(&self) -> String {
        self.hello().to_uppercase()
    }
}

struct English;
struct French;

impl Greet for English {
    fn hello(&self) -> String {
        String::from("hello")
    }
}

impl Greet for French {
    fn hello(&self) -> String {
        String::from("bonjour")
    }
    // shout() is inherited
}

fn main() {
    println!("{}", English.hello());     // hello
    println!("{}", French.shout());      // BONJOUR
}
```

Methods on a trait can have default implementations (like `shout` above) in terms of the other methods, so an `impl` only has to provide the mandatory pieces.

Using traits as bounds

Traits are how generics become useful. When you write `T: PartialOrd`, you are saying "T has to implement the `PartialOrd` trait." Methods from that trait are then available on any `T` that satisfies the bound.

Two syntactic forms of bounds, pick whichever is clearer:

```rust
fn print_it<T: std::fmt::Display>(x: T) {
    println!("{x}");
}

// Equivalent, but reads better with many bounds.
fn print_it_where<T>(x: T)
where T: std::fmt::Display
{
    println!("{x}");
}
```

You can combine bounds with `+`:

```rust
fn largest<T: PartialOrd + Copy>(xs: &[T]) -> T { ... }
```

Trait objects — `dyn Trait`

Sometimes you need a *collection* of values of different types that share a trait. Generics (monomorphisation) cannot express this: a `Vec<T>` has a single `T`. Trait objects — written `dyn Trait` — can:

```rust
trait Shape {
    fn area(&self) -> f64;
}

struct Circle { r: f64 }
struct Square { s: f64 }

impl Shape for Circle { fn area(&self) -> f64 { 3.14 * self.r * self.r } }
impl Shape for Square { fn area(&self) -> f64 { self.s * self.s } }

fn main() {
    let shapes: Vec<Box<dyn Shape>> = vec![
        Box::new(Circle { r: 1.0 }),
        Box::new(Square { s: 2.0 }),
    ];
    for s in &shapes {
        println!("{}", s.area());
    }
}
```

A `Box<dyn Shape>` is a fat pointer: one word for the data, one word for a *vtable* (a table of function pointers for this type's implementation of `Shape`). Calling `s.area()` is a virtual call through the vtable. It is slightly slower than generic dispatch — about the cost of one indirect jump — but sometimes that's the design you need.

Rule of thumb: use generics when you know the type at compile time (almost always); use `dyn Trait` when you genuinely need a heterogeneous collection or a pluggable interface.

Deriving traits

Many common traits have a macro that writes a sensible implementation for you:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
struct User {
    id: u32,
    name: String,
}

fn main() {
    let u = User { id: 1, name: String::from("Ada") };
    println!("{u:?}");              // Debug
    let v = u.clone();              // Clone
    assert_eq!(u, v);               // PartialEq
}
```

The most commonly derived traits are:

- `Debug` — `{:?}` formatting.
- `Clone` — explicit duplication via `.clone()`.
- `Copy` — implicit duplication on assignment (only for types whose every field is `Copy`).
- `PartialEq`, `Eq` — `==` and `!=`.
- `PartialOrd`, `Ord` — `<`, `>`, `<=`, `>=`.
- `Hash` — for use as a `HashMap` key.
- `Default` — provides `T::default()`.

If you want the derive's behaviour, `#[derive(...)]` is the one-line answer. If you need custom logic, implement the trait manually with `impl`.

Associated types

A trait may have *associated types* — type placeholders that are filled in by each implementation:

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}
```

This is the real signature of the core `Iterator` trait in the standard library. Each implementation picks an `Item` type:

```
struct Counter { n: u32 }

impl Iterator for Counter {
    type Item = u32;
    fn next(&mut self) -> Option<u32> {
        self.n += 1;
        if self.n <= 5 { Some(self.n) } else { None }
    }
}
```

Associated types versus generic parameters: the rule of thumb is *generic parameters when the caller chooses, associated types when the implementation chooses*. An iterator's `Item` type is determined by the iterator, not by the caller — so it is an associated type.

The orphan rule

You can only implement a trait for a type if **at least one** of the trait or the type is defined in your crate. This is the *orphan rule*, and it exists so that two unrelated crates cannot both implement the same trait for the same foreign type and silently conflict.

The workaround, when you want to add behaviour to a type from another crate, is the *newtype pattern*:

```rust
struct MyVec(Vec<i32>);        // wrap the foreign type

impl std::fmt::Display for MyVec {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "[{}]", self.0.iter().map(|n| n.to_string())
                                        .collect::<Vec<_>>()
                                        .join(", "))
    }
}
```

Because `MyVec` is defined here, the orphan rule is satisfied, and you can implement any trait you like on it.

5.5 Worked example — a small parser error

Here is everything in this chapter, working together, in a small example. We'll build a type for the result of parsing an integer from a string, with a custom error.

```rust
#[derive(Debug, PartialEq)]
enum ParseError {
    Empty,
    NonDigit(char),
    Overflow,
}

#[derive(Debug, PartialEq)]
struct Parsed {
    value: i64,
    consumed: usize,
}

fn parse_int(s: &str) -> Result<Parsed, ParseError> {
    if s.is_empty() {
        return Err(ParseError::Empty);
    }
    let mut value: i64 = 0;
    let mut consumed = 0;
    for c in s.chars() {
        match c.to_digit(10) {
            Some(d) => {
                value = value
                    .checked_mul(10)
                    .and_then(|v| v.checked_add(d as i64))
                    .ok_or(ParseError::Overflow)?;
                consumed += 1;
            }
            None => {
                if consumed == 0 {
                    return Err(ParseError::NonDigit(c));
                }
                break;
            }
        }
    }
    Ok(Parsed { value, consumed })
}

fn main() {
    println!("{:?}", parse_int("42 apples"));
    println!("{:?}", parse_int("abc"));
    println!("{:?}", parse_int(""));
}
```

Output:

```
Ok(Parsed { value: 42, consumed: 2 })
Err(NonDigit('a'))
Err(Empty)
```

Count the tools we used:

- **A struct** (`Parsed`) to name the two outputs.
- **An enum** (`ParseError`) to enumerate failure modes.
- `derive(Debug, PartialEq)` so we can print and compare.
- **Generics** — `Result<T, E>` is generic, and we instantiated it with our own types.
- **Pattern matching** on the `Option` that `c.to_digit(10)` returns.
- **The** `?` **operator** for error propagation (Chapter 6).

This is the core pattern of a Rust codebase: data described as structs and enums, behaviour expressed as methods and traits, errors as values. The next chapter dives deeper into the last of those.

Next: Chapter 6 — Error Handling

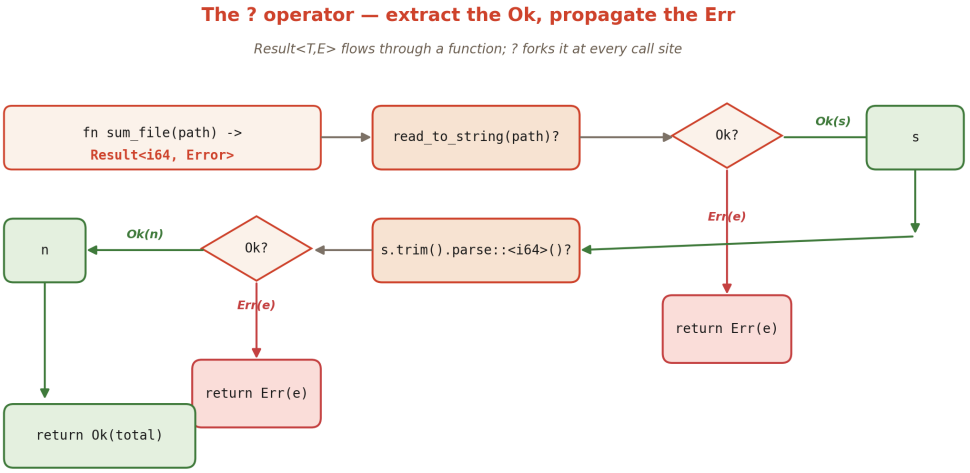
CHAPTER

Error Handling

Rust does not have exceptions. This is a design choice that, more than any other, gives the language its particular *honest* feel: whatever can fail has that possibility encoded in its return type, right there in the signature, and you are *obligated* to deal with it before you can go on to the next line.

There is nothing mysterious about how it works. Two enums — `Option<T>` for "might be absent" and `Result<T, E>` for "might have failed" — carry the bad news. One operator — `?` — propagates it. The whole error-handling machinery is built on pieces you already met in Chapter 5. The interest is in learning to use them well.

6.1 Two kinds of failure



`Result` threads through a function until `?` extracts the success value or returns the error.

Rust draws a bright line between two things most languages smear together:

Unrecoverable errors — panics. The program is in a state it cannot continue from safely: an assertion failed, an array index was out of bounds, a required invariant was violated. Panics unwind the stack, run destructors along the way, and terminate the thread. You rarely *catch* a panic; you fix the code that caused it.

Recoverable errors — Results. Something went wrong, but the program can handle it. A file was missing, a network call timed out, a user typed nonsense into a form. These are not bugs; they are expected outcomes, represented as values, and the code that calls the fallible function gets to decide what to do.

Most application code should produce `Result<T, E>` for its fallible operations and leave panics for genuine invariant violations. The two APIs are not interchangeable; use the right tool.

6.2 Option

`Option<T>` is the type for values that might be absent:

```
enum Option<T> {
    Some(T),
    None,
}
```

It replaces nullable pointers. In Rust, you cannot have a `&String` that is null; if you need "a string or nothing," you use `Option<String>`, and the compiler makes you check which one before you can use it.

A function that looks up a value by key returns `Option`:

```
fn find<'a>(map: &'a [(String, i32)], key: &str) -> Option<&'a i32> {
    for (k, v) in map {
        if k == key { return Some(v); }
    }
    None
}
```

And the caller must decide what to do with the two cases:

```
let data = vec![
    ("one".to_string(), 1),
    ("two".to_string(), 2),
];

match find(&data, "two") {
    Some(&n) => println!("found {n}"),
    None => println!("not found"),
}
```

Unwrapping

`Option` comes with a rich family of methods. The handful you will use most:

```
let a: Option<i32> = Some(5);
let b: Option<i32> = None;

a.unwrap();                 // 5; panics if None
b.unwrap_or(0);             // 0
b.unwrap_or_else(|| compute_default());
a.map(|x| x * 2);           // Some(10)
a.and_then(|x| if x > 0 { Some(x) } else { None });
a.ok_or("missing");         // Result<i32, &str>
```

`unwrap()` is the sharp tool. It succeeds when the value is `Some(x)` and panics when it is `None`. You are saying "I am sure this is not `None`, and if I am wrong I want to crash." That is sometimes exactly right — in tests, in prototypes, in the main function of a small script — and sometimes exactly wrong. Default to one of the other methods; reach for `unwrap` deliberately.

> **Tip**
> `expect("reason")` is like `unwrap()` but panics with a custom message. Use it instead of `unwrap()` whenever you can; the message will save you time when the assertion eventually fails.

6.3 Result

`Result<T, E>` carries either a success value or an error:

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

The fallible standard-library APIs return `Result`:

```
use std::fs;

let text = fs::read_to_string("config.toml");
match text {
    Ok(s) => println!("read {} bytes", s.len()),
    Err(e) => eprintln!("could not read config: {e}"),
}
```

`Result` has the same family of methods as `Option` and more:

```
let r: Result<i32, String> = Ok(5);
r.unwrap();                  // 5
r.unwrap_or(0);
r.map(|x| x + 1);            // Ok(6)
r.map_err(|e| format!("oops: {e}"));
r.and_then(|x| if x > 0 { Ok(x) } else { Err("neg".into()) });
r.ok();                      // Option<i32>, discards the error
```

6.4 The `?` operator

`match` on every fallible call would produce pyramids of code. The `?` operator flattens them:

```
use std::fs;
use std::io;

fn word_count(path: &str) -> io::Result<usize> {
    let text = fs::read_to_string(path)?;       // early-return if Err
    Ok(text.split_whitespace().count())
}
```

Read `text?` as "if this is `Ok(x)`, give me `x` and continue; if this is `Err(e)`, return `Err(e)` from the enclosing function." It is the same code that a `match` would produce, just much quieter:

```
// Without ?
let text = match fs::read_to_string(path) {
    Ok(s) => s,
    Err(e) => return Err(e),
};
```

`?` works on `Option` too (it returns `None` from the enclosing function), but the function's return type has to match.

Error conversion with From

The interesting case is when `?` is used on a `Result<_, E1>` but the enclosing function returns `Result<_, E2>`. The compiler inserts an automatic conversion *if* there is a `From<E1> for E2` implementation:

```
use std::fs;
use std::io;
use std::num::ParseIntError;

#[derive(Debug)]
enum AppError {
    Io(io::Error),
    Parse(ParseIntError),
}

impl From<io::Error> for AppError {
    fn from(e: io::Error) -> Self { AppError::Io(e) }
}

impl From<ParseIntError> for AppError {
    fn from(e: ParseIntError) -> Self { AppError::Parse(e) }
}

fn read_number(path: &str) -> Result<i64, AppError> {
    let text = fs::read_to_string(path)?;          // io::Error -> AppError
    let n: i64 = text.trim().parse()?;             // ParseIntError -> AppError
    Ok(n)
}
```

This pattern — a crate-level enum whose variants wrap the errors of all the libraries you call into — is the idiomatic way to present a unified error surface to your users.

6.5 Custom errors

A useful error type usually has:

- `#[derive(Debug)]` so `{:?}` works.
- An implementation of `Display` for human-readable messages.
- An implementation of `std::error::Error` for interoperability with other error-handling code.

Written by hand:

```rust
use std::fmt;

#[derive(Debug)]
pub enum ConfigError {
    Missing(String),
    Malformed { line: usize, reason: String },
}

impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            ConfigError::Missing(k) => write!(f, "missing key `{k}`"),
            ConfigError::Malformed { line, reason } =>
                write!(f, "line {line}: {reason}"),
        }
    }
}

impl std::error::Error for ConfigError {}
```

That works, but it is tedious. In practice everyone uses a crate called `thiserror` to generate the boilerplate:

```rust
use thiserror::Error;

#[derive(Debug, Error)]
pub enum ConfigError {
    #[error("missing key `{0}`")]
    Missing(String),
    #[error("line {line}: {reason}")]
    Malformed { line: usize, reason: String },
    #[error("could not read config: {0}")]
    Io(#[from] std::io::Error),
}
```

The `#[from]` attribute generates the `From<io::Error> for ConfigError` implementation, so `?` will do the right thing on a `std::io::Result`.

6.6 `anyhow` for application code

Libraries should define precise error types that name every variant (using `thiserror` or a hand-rolled enum). Applications often don't care about the exact shape of the error — they care about getting it to the log with a useful message and a backtrace. The `anyhow` crate is built for that.

```
use anyhow::{Context, Result};

fn load_user(id: u64) -> Result<User> {
    let path = format!("users/{id}.json");
    let text = std::fs::read_to_string(&path)
        .with_context(|| format!("reading {path}"))?;
    let user: User = serde_json::from_str(&text)
        .with_context(|| format!("parsing {path}"))?;
    Ok(user)
}
```

`anyhow::Result<T>` is an alias for `Result<T, anyhow::Error>`, where `anyhow::Error` is a type-erased wrapper that can hold *any* error. The `.with_context(|| …)` extension adds a layer of explanation at each `?`, so when the error eventually surfaces you get something like:

```
Error: parsing users/42.json

Caused by:
    0: reading users/42.json
    1: No such file or directory (os error 2)
```

The division of labour is:

- **Library code** → `thiserror`, concrete enums. Callers can match on the error kind and react.
- **Application code** → `anyhow`. Propagate with context; log the chain at the top level.

A single crate can be both; just don't leak `anyhow::Error` out of your library's public API.

6.7 When to panic

Panicking is appropriate when:

- **An invariant has been violated.** `vec[17]` on a vector of length 5 panics because the program is broken, and there is no sensible recovery.
- **You are prototyping.** `.unwrap()` everywhere is fine for a ten-line script; you will replace the important ones with `?` when the code graduates to production.
- **You wrote a bug.** `assert!(invariant)` is a form of panic that catches the bug early.

Panicking is inappropriate when:

- **The failure comes from outside your program.** Missing files, network errors, malformed user input — these are not bugs. Return `Result`.
- **You're in library code.** Library callers should get to decide how to handle errors; don't panic on their behalf.

If you're unsure, return `Result`. You can always `unwrap` a `Result` at the call site; you cannot `unpanic` a panic.

6.8 Catching panics

For the rare case when you *do* need to catch a panic — usually a worker thread or an FFI boundary — there is `std::panic::catch_unwind`:

```
use std::panic;

let result = panic::catch_unwind(|| {
    let v: Vec<i32> = vec![];
    v[0]                          // panics
});

match result {
    Ok(_) => println!("ok"),
    Err(_) => println!("caught a panic"),
}
```

This is not a substitute for proper error handling. It is a tool for isolating failures at boundaries, and it comes with caveats — the closure must not be holding locks in an inconsistent state, the payload is `Box<dyn Any>` which is ugly to introspect, and panics can be configured to abort the process instead of unwinding, in which case `catch_unwind` does nothing. Use it sparingly.

6.9 Main can return a Result

A convenient pattern for CLI tools: have `main` itself return a `Result`. The runtime prints the error (via `Debug`) and exits with a non-zero status for you.

```
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let config = std::fs::read_to_string("config.toml")?;
    run(&config)?;
    Ok(())
}
```

Or, with `anyhow`:

```
fn main() -> anyhow::Result<()> {
    let config = std::fs::read_to_string("config.toml")
        .context("reading config.toml")?;
    run(&config)?;
    Ok(())
}
```

No `unwrap`, no `match`, no `exit(1)`. Clean top-level code.

6.10 A complete picture

Here is a small program that ties everything together — `Option`, `Result`, `?`, custom errors, and `main`-returns-Result:

```
use std::fs;
use std::num::ParseIntError;
use thiserror::Error;

#[derive(Debug, Error)]
enum Error {
    #[error("io: {0}")]
    Io(#[from] std::io::Error),
    #[error("parse: {0}")]
    Parse(#[from] ParseIntError),
    #[error("empty file")]
    Empty,
}

fn sum_file(path: &str) -> Result<i64, Error> {
    let text = fs::read_to_string(path)?;
    let mut total: i64 = 0;
    let mut any = false;
    for line in text.lines() {
        let line = line.trim();
        if line.is_empty() { continue; }
        total += line.parse::<i64>()?;
        any = true;
    }
    if !any { return Err(Error::Empty); }
    Ok(total)
}

fn main() -> Result<(), Error> {
    let total = sum_file("numbers.txt")?;
    println!("total = {total}");
    Ok(())
}
```

Read the `sum_file` signature: `Result<i64, Error>`. Everything that follows is constrained by that choice. `?` on the file read turns an `io::Error` into our `Error::Io`. `?` on the parse turns a `ParseIntError` into `Error::Parse`. The `Error::Empty` case is one we invented, for the one failure mode that the standard library doesn't already name.

This is the whole shape of error handling in Rust. You will recognise it everywhere.

Next: Chapter 7 — Collections and Iterators

CHAPTER

Collections and Iterators

Every real program stores collections and walks through them. In Rust, the two things go together so tightly that once you internalise the iterator model, you will catch yourself writing entire functions that are a single iterator chain with no intermediate variables — and that chain will compile to the same assembly as the hand-rolled `for` loop you would have written in C.

This chapter is in two halves. The first names the collections you reach for most. The second explains the iterator protocol that makes them a joy to use.

7.1 Vec

`Vec<T>` is the workhorse growable array. It stores `T` values in a contiguous heap buffer, grows automatically when full, and can be pushed to, popped from, indexed, sliced, and iterated.

```
let mut v: Vec<i32> = Vec::new();       // empty
v.push(10);
v.push(20);
v.push(30);

let w = vec![1, 2, 3, 4, 5];            // literal macro
let zeros = vec![0; 16];                // sixteen zeros

println!("{}", v[1]);                    // 20 — panics if out of range
println!("{:?}", v.get(99));             // None — bounds-checked

for x in &v {
    println!("{x}");
}
```

The two indexing forms are deliberate. `v[i]` is convenient and panics on out-of-bounds — use it when the index is known-good by construction. `v.get(i)` returns `Option<&T>` — use it when the index might be invalid and you want to handle the case.

Common Vec methods:

```
v.len();                 // number of elements
v.is_empty();
v.push(x);               // append (may reallocate)
v.pop();                 // Option<T>
v.insert(i, x);          // shift right — O(n)
v.remove(i);             // shift left — O(n)
v.swap_remove(i);        // O(1), but reorders
v.clear();
v.sort();                // T: Ord required
v.sort_by_key(|s| s.len());
v.dedup();               // remove consecutive duplicates
v.extend([9, 10, 11]);
```

A `Vec<T>` can hand out slices into itself:

```
let v = vec![10, 20, 30, 40, 50];
let slice: &[i32] = &v[1..4];            // &[20, 30, 40]
```

Slices are what most functions should take in their argument list — `&[T]` or `&mut [T]` — because a function that accepts a slice can be called with a vector, an array, or any other slice without copying.

> **Tip**
>
> **Capacity.** A `Vec` has two lengths: `len()` (how many elements are in it) and `capacity()` (how many fit before it needs to reallocate). Calling `Vec::with_capacity(n)` reserves room for `n` elements up front, which avoids reallocations if you know the size.

7.2 String

`String` is the growable, owned, heap-allocated, guaranteed-UTF-8 string. Its borrowed counterpart is `&str`. The relationship is exactly the same as `Vec<T>` / `&[T]`:

```
let s: String = String::from("hello");
let s2: String = "world".to_string();
let owned: String = format!("{s}, {s2}");

let borrowed: &str = "hello";            // string literal — lives in binary
let slice: &str = &owned[0..5];          // slice into the String
```

The one surprise is that you **cannot index a string with** `s[i]`:

```
let s = String::from("résumé");
// let c = s[2];                          // ERROR: can't index str
```

Why not? Because `s` is UTF-8-encoded. `s[2]` could sensibly mean "the third *byte*" or "the third *character*" or "the third *grapheme cluster*", and these are three different things once non-ASCII characters arrive. Rust refuses to guess. Instead:

```
for byte in s.bytes()  { /* u8 */  }      // raw bytes
for ch in s.chars()    { /* char */ }     // Unicode scalar values
for (i, c) in s.char_indices() { /* byte offset + char */ }
```

Most of the time you want `chars()` for a character-by-character view or `bytes()` for raw processing. For "what the user sees" you want *grapheme clusters*, which are not in the standard library — reach for the `unicode-segmentation` crate.

7.3 HashMap and HashSet

`HashMap<K, V>` is the hash-based key→value map. The key type must implement `Eq + Hash`.

```
use std::collections::HashMap;

let mut scores: HashMap<String, i32> = HashMap::new();
scores.insert("Ada".to_string(), 95);
scores.insert("Grace".to_string(), 99);

if let Some(s) = scores.get("Ada") {
    println!("Ada: {s}");
}

for (name, score) in &scores {
    println!("{name}: {score}");
}
```

The `entry` API is the idiomatic way to update-or-insert:

```rust
let mut counts: HashMap<&str, i32> = HashMap::new();
for word in ["apple", "banana", "apple", "pear", "banana", "apple"] {
    *counts.entry(word).or_insert(0) += 1;
}
println!("{:?}", counts);        // {"apple": 3, "banana": 2, "pear": 1}
```

`entry(key)` returns a handle into the map at that key; `.or_insert(0)` says "if there's no value there, put 0"; the `*` dereferences the `&mut i32` we got back so we can add to it.

`HashSet<T>` is the key-only cousin — a hash-based unique collection:

```rust
use std::collections::HashSet;

let mut seen: HashSet<i32> = HashSet::new();
for n in [1, 2, 3, 2, 1, 4] {
    if !seen.insert(n) {
        println!("duplicate: {n}");
    }
}
```

Rust's default `HashMap` uses a randomly-seeded hash function (SipHash) for protection against HashDoS attacks. If you have trusted keys and need raw speed, swap in a faster hasher via the `hashbrown` or `ahash` crate.

7.4 BTreeMap and BTreeSet

When you need **sorted** iteration, use the B-tree variants:

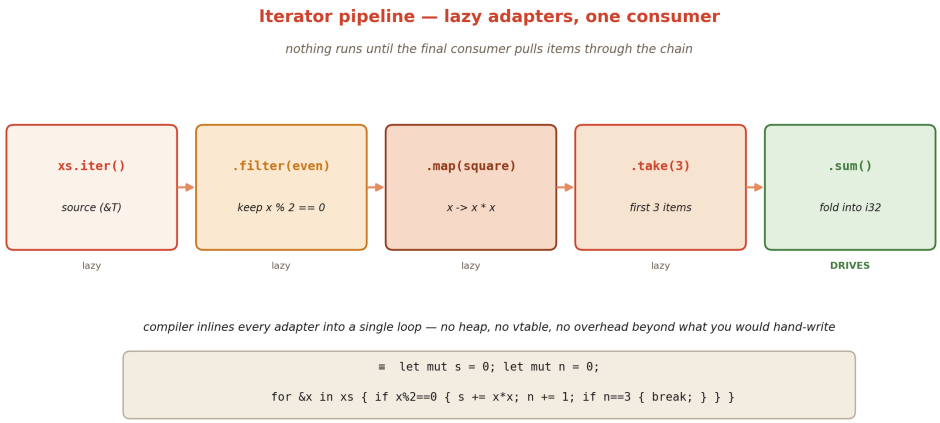
```rust
use std::collections::BTreeMap;

let mut grades = BTreeMap::new();
grades.insert("Ada", 95);
grades.insert("Bob", 72);
grades.insert("Carl", 88);

for (name, grade) in &grades {         // alphabetical, always
    println!("{name}: {grade}");
}
```

`BTreeMap` has the same API as `HashMap` but orders keys by their `Ord` implementation. Use it when order matters (printing, range queries) or when keys are not hashable. Its lookups are `O(log n)` rather than `O(1) amortised`, but in practice for small maps the difference is invisible.

7.5 The Iterator trait



Iterator adapters build a pipeline that executes lazily; the `collect` at the end is what drives the chain.

`Iterator` is the trait at the centre of the standard library:

```
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;

    // ... hundreds of default methods, all built on next() ...
}
```

A type is an iterator if it can produce its next item on demand. Everything iterable — vectors, strings, ranges, file lines, hashmap entries, even ASCII characters — either *is* an iterator or can hand one out.

Three methods turn something into an iterator:

```
let v = vec![1, 2, 3];

for x in v.iter()       { /* x: &i32  — borrow each element */ }
for x in v.iter_mut()   { /* x: &mut i32 — mutable borrow */ }
for x in v.into_iter()  { /* x: i32   — consumes the Vec */ }
```

`for x in v` is sugar for `for x in v.into_iter()`. That is why a bare `for x in v` *moves* the vector — and why you usually write `for x in &v` (which is sugar for `v.iter()`) when you just want to look.

7.6 Adapters and consumers

Iterators are composed of *adapters* (which return a new iterator) and *consumers* (which actually do work).

Common adapters, each one lazy:

```
let xs = vec![1, 2, 3, 4, 5, 6];

let _ = xs.iter().map(|x| x * x);            // square each
let _ = xs.iter().filter(|&&x| x % 2 == 0);  // even only
let _ = xs.iter().take(3);                   // first three
let _ = xs.iter().skip(2);                   // all but first two
let _ = xs.iter().rev();                     // reversed
let _ = xs.iter().enumerate();               // (0, &1), (1, &2), ...
let _ = xs.iter().zip([10, 20, 30].iter());  // pair them up
let _ = xs.iter().chain([99].iter());        // concatenate
```

Each `_ = …` here is an iterator. *Nothing has actually run.* An iterator is a *description* of work to do. The work happens when a consumer pulls values out.

Consumers:

```rust
let xs = vec![1, 2, 3, 4, 5, 6];

let sum: i32 = xs.iter().sum();                    // 21
let product: i32 = xs.iter().product();            // 720
let count = xs.iter().filter(|&&x| x > 3).count(); // 3
let max = xs.iter().max();                         // Some(&6)

let squares: Vec<i32> = xs.iter().map(|x| x * x).collect();
let seen: std::collections::HashSet<i32> =
    xs.iter().copied().collect();

for (i, x) in xs.iter().enumerate() {
    println!("{i}: {x}");
}
```

`for` is itself a consumer. So is `collect`, `sum`, `count`, `max`, `fold`, `find`, `any`, `all`, `reduce`. When you see one of these at the end of a chain, the pipeline runs.

Collect and FromIterator

`collect` is polymorphic over its output. Any type that implements `FromIterator<T>` can be the target — and you choose which via type annotation or turbofish:

```rust
let v: Vec<i32> = (0..5).collect();
let s: String = ['h', 'i'].into_iter().collect();
let m: std::collections::HashMap<i32, i32> =
    (0..5).map(|x| (x, x * x)).collect();

// Or via turbofish:
let v = (0..5).collect::<Vec<i32>>();
```

`Result` has a convenient `FromIterator` impl: an iterator of `Result<T, E>` collects into `Result<Vec<T>, E>` — if *any* item is `Err`, the collect is short-circuited and returns that error.

```rust
let parsed: Result<Vec<i32>, _> = ["1", "2", "3"].iter()
    .map(|s| s.parse::<i32>())
    .collect();
// Ok(vec![1, 2, 3])

let bad: Result<Vec<i32>, _> = ["1", "oops", "3"].iter()
    .map(|s| s.parse::<i32>())
    .collect();
// Err(...)
```

7.7 Why iterators are zero-cost

Here is an iterator chain:

```rust
fn sum_of_squares_of_evens(xs: &[i32]) -> i32 {
    xs.iter()
      .filter(|&&x| x % 2 == 0)
      .map(|x| x * x)
      .sum()
}
```

And here is the "manual" version:

```rust
fn sum_of_squares_of_evens_manual(xs: &[i32]) -> i32 {
    let mut total = 0;
    for &x in xs {
        if x % 2 == 0 {
            total += x * x;
        }
    }
    total
}
```

These compile to **the same machine code**. There is no callback overhead, no dynamic dispatch, no heap allocation. Every iterator adapter is a zero-sized wrapper struct; every closure is inlined; `sum` is a `fold` that the optimiser unrolls. The high-level version is not a performance concession — it is a stylistic choice.

7.8 Closures

Iterator chains lean heavily on closures. Three syntactic forms:

```
let add_one = |x: i32| x + 1;                 // expression body
let square = |x: i32| -> i32 { x * x };       // block body with return type

let mut xs = vec![1, 2, 3];
xs.iter_mut().for_each(|x| *x *= 10);
```

Closures capture variables from the enclosing scope. The compiler figures out how they capture — by reference, by mutable reference, or by move — based on how you use them inside the closure. You can force a move capture with the `move` keyword:

```
let data = vec![1, 2, 3];
let handle = std::thread::spawn(move || {
    println!("{:?}", data);           // `data` is moved into the thread
});
handle.join().unwrap();
```

Under the hood, closures are anonymous structs that capture their environment as fields plus an `impl` of one of three traits:

- `Fn` — can be called by reference; captures by `&`.
- `FnMut` — can be called by mutable reference; captures by `&mut`.
- `FnOnce` — can be called once, and consumes captured values.

You usually don't think about this. When you write a higher-order function that takes a closure, you pick the bound that expresses the least power you need:

```
fn call_once<F: FnOnce()>(f: F) { f(); }
fn call_many<F: Fn()>(f: F) { for _ in 0..3 { f(); } }
```

7.9 A small pipeline

Let us end with a worked example. Read a file of CSV-shaped numbers — one record per line, comma-separated — and report the per-column mean:

```rust
use std::fs;
use std::io;

fn column_means(path: &str) -> io::Result<Vec<f64>> {
    let text = fs::read_to_string(path)?;

    let rows: Vec<Vec<f64>> = text
        .lines()
        .filter(|l| !l.trim().is_empty())
        .map(|l| l
            .split(',')
            .map(|c| c.trim().parse::<f64>().unwrap_or(0.0))
            .collect::<Vec<f64>>())
        .collect();

    if rows.is_empty() {
        return Ok(vec![]);
    }

    let ncols = rows[0].len();
    let mut sums = vec![0.0; ncols];
    for row in &rows {
        for (j, &v) in row.iter().enumerate() {
            sums[j] += v;
        }
    }

    let nrows = rows.len() as f64;
    let means: Vec<f64> = sums.iter().map(|s| s / nrows).collect();
    Ok(means)
}

fn main() -> io::Result<()> {
    let means = column_means("data.csv")?;
    for (i, m) in means.iter().enumerate() {
        println!("column {i}: mean = {m:.3}");
    }
    Ok(())
}
```

Notice the compound structure of the outer iterator: the closure passed to `map` *itself* parses each line into a `Vec<f64>` by building and collecting an inner iterator. Annotations like `::<Vec<f64>>` (the turbofish) tell `collect` which collection to build when the context does not make it obvious.

This is the shape of most Rust data-processing code. You will find yourself reaching for it again and again — and you will be surprised how often a pipeline like this can be written without a single intermediate `let` or temporary buffer, while still being as fast as anything you could write by hand.

Next: Chapter 8 — Concurrency and Async

CHAPTER

Concurrency and Async

Rust's claim to *fearless concurrency* sounds like marketing. It isn't. The same ownership rules that prevent use-after-free in single-threaded code prevent **data races** in multi-threaded code, at compile time, without runtime overhead. If your program compiles and uses only safe Rust, you cannot have a data race. Period.

This does not mean Rust programs cannot have concurrency bugs. Deadlocks are still possible. Logic races — "the two threads both did the sensible thing, but not in the order I wanted" — are still possible. Performance pathologies are still possible. But the category of bug that has historically cost systems programmers the most wall-clock time to debug — two threads touching the same memory without synchronisation, producing a value neither one intended — is *gone*.

This chapter has two halves: **threads** (OS-level, preemptive concurrency) and **async** (cooperative, single- or multi-threaded concurrency for I/O). They solve different problems; most real applications use both.

8.1 Spawning threads

`std::thread::spawn` starts a new OS thread:

```rust
use std::thread;
use std::time::Duration;

fn main() {
    let handle = thread::spawn(|| {
        for i in 0..5 {
            println!("[spawned] {i}");
            thread::sleep(Duration::from_millis(50));
        }
    });

    for i in 0..3 {
        println!("[main] {i}");
        thread::sleep(Duration::from_millis(70));
    }

    handle.join().unwrap();
}
```

`spawn` returns a `JoinHandle<T>`; calling `.join()` blocks until the thread finishes and returns its value (or the panic payload if it panicked). If you drop the handle without joining, the main thread does not wait — if `main` exits first, the spawned thread is terminated with it.

Moving data into a thread

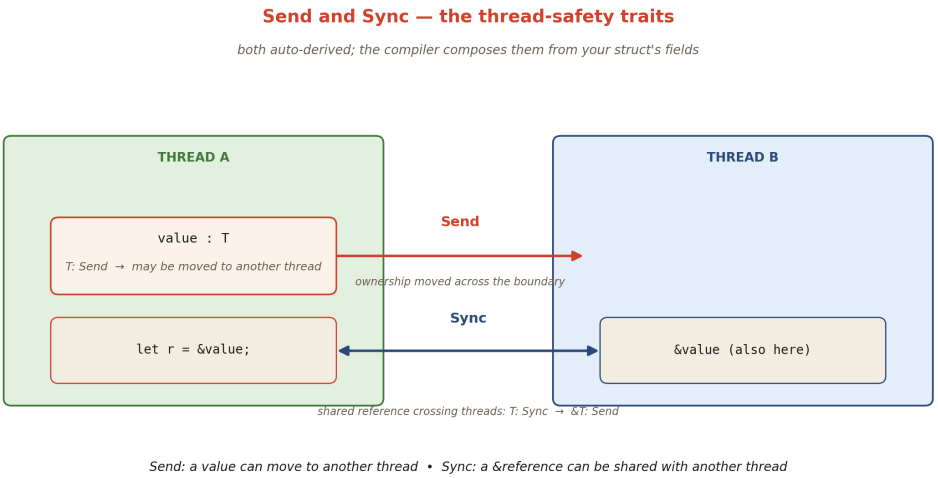
The closure passed to `spawn` must be `'static` (it may outlive the function that spawned it) and `Send` (it may run on another thread). In practice this means you have to `move` owned data in:

```rust
let v = vec![1, 2, 3];

let handle = thread::spawn(move || {
    println!("{:?}", v);           // v is now owned by the thread
});
// println!("{:?}", v);            // ERROR: v was moved
handle.join().unwrap();
```

Without `move`, the closure would try to borrow `v`, but the borrow could outlive `main`'s stack frame. The compiler refuses to compile that — and by doing so, it prevents a dangling-pointer-across-threads bug that is nearly impossible to reproduce on demand in C.

8.2 Send and Sync



`Send` means a value can be moved to another thread. `Sync` means `&T` can be shared between threads. The compiler inserts these bounds automatically.

Two auto-traits in the standard library encode the compiler's thread-safety rules:

- `Send` — a type is `Send` if it is safe to *move* a value of that type to another thread. Almost every type is. Exceptions are things like `Rc<T>` (non-atomic refcount) and raw pointers.
- `Sync` — a type is `Sync` if it is safe to share a `&T` reference between threads. Equivalently, `T: Sync` iff `&T: Send`.

`thread::spawn` requires `F: Send + 'static` on its closure. Everything the closure captures must be `Send`. If you try to capture a non-`Send` type, the compiler explains in detail:

```
error[E0277]: `Rc<String>` cannot be sent between threads safely
  --> src/main.rs:5:18
   |
 5 |     thread::spawn(move || println!("{rc}"));
   |     ------------ ^^^^^^^^^^^^^^^^^^^^^^^^^^
   |     |
   |     within this `[closure]`, the trait `Send` is not
   |     implemented for `Rc<String>`
   |
help: use `Arc` instead of `Rc`
```

The compiler is not just rejecting your code; it is telling you the fix. `Rc<T>` is for single-threaded reference counting; `Arc<T>` is its thread-safe cousin (the "A" is for *atomic*).

You never implement `Send` or `Sync` manually. They are *auto- traits* — the compiler derives them based on your fields. If you build a struct out of `Send` parts, it is `Send`; out of `Sync` parts, `Sync`. Fearless concurrency means *you do not mark your own types*, and yet the guarantees flow through.

8.3 Sharing data — Arc and Mutex

The rule from Chapter 4 — "one mutable reference or many shared references, never both" — is what makes single-threaded Rust safe. For multi-threaded code, we need runtime checking because the compiler cannot know which threads are running when. The two tools that provide it are:

- `Arc<T>` — atomic reference counting, for *shared ownership* across threads. Increments and decrements are atomic.
- `Mutex<T>` — a mutual-exclusion lock, for *exclusive access* to the wrapped data at runtime.

Combined: `Arc<Mutex<T>>` is the idiomatic "shared mutable state" building block.

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut guard = counter.lock().unwrap();
            *guard += 1;
        });
        handles.push(handle);
    }

    for h in handles { h.join().unwrap(); }
    println!("counter = {}", *counter.lock().unwrap());   // 10
}
```

A few things to notice:

- `Arc::clone(&counter)` creates a new *handle* to the same data; the refcount goes up. Each thread receives its own clone.
- `counter.lock()` returns a `LockResult<MutexGuard<T>>`. The `guard` is a smart pointer that you can deref to get at the inner value, and that releases the lock when it is dropped. The `unwrap()` is there because a mutex can be *poisoned* if a prior lock-holder panicked; in production you often want to handle that case explicitly.
- The lock is **scoped to the guard's lifetime**. No forgotten unlocks. No double-unlocks. If you panic while holding the lock, RAII still releases it on the way out.

Inside `spawn`, the closure captures `counter` by `move`. If you tried to use the original `counter` *without* cloning it first, each iteration of the loop would move the same value, and the second iteration would fail to compile.

RwLock

When reads heavily outnumber writes, an `RwLock` can be faster. It allows many concurrent readers *or* one exclusive writer:

```
use std::sync::RwLock;

let cache: RwLock<HashMap<String, String>> = RwLock::new(HashMap::new());

// Many threads read concurrently:
let r = cache.read().unwrap();
if let Some(v) = r.get("key") { /* ... */ }
drop(r);

// One thread writes at a time:
let mut w = cache.write().unwrap();
w.insert("key".into(), "value".into());
```

The tradeoff: `RwLock` is more expensive in the no-contention case than `Mutex`. Profile before you switch.

8.4 Channels

`Mutex` is the low-level tool. `Channel` is usually the higher-level one you want: a lock-free FIFO that moves ownership of a value from producer to consumer.

```
use std::sync::mpsc;     // multi-producer, single-consumer
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    for i in 0..3 {
        let tx = tx.clone();
        thread::spawn(move || {
            tx.send(format!("hello from {i}")).unwrap();
        });
    }
    drop(tx);     // close the sender so the receiver finishes

    for msg in rx {
        println!("{msg}");
    }
}
```

The channel owns the values in flight. `tx.send(x)` transfers ownership of `x` into the channel; `rx.recv()` takes ownership out. There is no shared mutable state, no lock to

forget — just move semantics extended across thread boundaries.

For real applications you'll often prefer the `crossbeam::channel` crate, which is faster and also supports the *multi-consumer* case (`mpmc`).

8.5 Atomics

For the simplest kind of shared state — counters, flags, single machine-word values — locks are overkill. The `std::sync::atomic` module exposes lock-free atomic types:

```
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::Arc;
use std::thread;

let count = Arc::new(AtomicU64::new(0));
let mut handles = vec![];

for _ in 0..10 {
    let c = Arc::clone(&count);
    handles.push(thread::spawn(move || {
        for _ in 0..1_000 {
            c.fetch_add(1, Ordering::Relaxed);
        }
    }));
}
for h in handles { h.join().unwrap(); }
println!("{}", count.load(Ordering::Relaxed));   // 10000
```

`Ordering::Relaxed` says "atomic, but no ordering with respect to other memory operations." That is exactly what a counter needs. For synchronisation between threads (publishing a flag, handing off a pointer), you want `Acquire` / `Release` or `SeqCst`, and you want to read a book about memory ordering before you deploy it. When in doubt, use a `Mutex`.

8.6 Rayon — effortless data parallelism

The `rayon` crate turns sequential iterator chains into parallel ones by changing one word:

```
use rayon::prelude::*;

let sum: u64 = (1u64..=1_000_000).into_par_iter().sum();
```

`par_iter()` and friends distribute the work across a thread pool. The rest of the pipeline — `map`, `filter`, `sum`, `collect` — all work exactly as they do on a normal iterator.

Rayon is what you reach for whenever you have CPU-bound work over a collection. It is safe, correct, and often delivers 4× speedup on a 4-core laptop with zero code change beyond the prelude.

8.7 Async — when blocking is the problem

Threads are great when the work is *CPU-bound* — you want to use all your cores. They are expensive when the work is *I/O-bound* and you need *many* concurrent operations — hundreds of open sockets, thousands of pending HTTP requests — because each thread costs memory (stacks are megabytes) and context-switching isn't free either.

Rust's `async` / `await` gives you *cooperative* concurrency: many concurrent tasks multiplexed onto a small number of threads, with the compiler generating the bookkeeping.

The shape of an async function

```
async fn fetch(url: &str) -> reqwest::Result<String> {
    let body = reqwest::get(url).await?.text().await?;
    Ok(body)
}
```

What `async fn` actually returns is not a `String`; it is a `Future<Output = reqwest::Result<String>>`. A future is a state machine the compiler generated for you. Until someone *polls* the future, nothing runs. Polling happens inside an **executor**.

Executors

The standard library has `async` syntax but no executor. You bring one in as a dependency. The ubiquitous choice is `tokio`:

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let body = fetch("https://example.com").await?;
    println!("{}", &body[..80]);
    Ok(())
}
```

The `#[tokio::main]` attribute wraps `main` in a synchronous bootstrap that starts the tokio runtime and drives the returned future to completion.

Awaiting and concurrency

`await` suspends the current task until the future it is awaiting is ready. Other tasks on the same executor can run in the meantime.

To run multiple futures *concurrently*, don't just await them in sequence — that serializes them. Use `tokio::join!` or `futures::join_all`:

```
use tokio::time::{sleep, Duration};

async fn work(id: u32) {
    println!("{id} start");
    sleep(Duration::from_millis(500)).await;
    println!("{id} done");
}

#[tokio::main]
async fn main() {
    // Serial — total ~1500ms:
    work(1).await;
    work(2).await;
    work(3).await;

    // Concurrent — total ~500ms:
    tokio::join!(work(4), work(5), work(6));
}
```

The first three run one after the other. The last three run concurrently on the tokio runtime — same thread, but each task yields while sleeping, so they overlap.

For a *dynamic* number of tasks, spawn them:

```
let mut handles = vec![];
for i in 0..100 {
    handles.push(tokio::spawn(async move {
        // do some work ...
        i * 2
    }));
}
let results: Vec<_> = futures::future::join_all(handles).await;
```

The colour problem

Async functions can only be awaited from inside other async functions. This is called the *function colour* problem: `async fn` is one colour, regular `fn` is another, and you cannot call blue from red without help. Rust is not unique in this; JavaScript, Python, and C# all have the same issue.

The practical consequences:

- Libraries often have to pick: sync API or async API, and the two are not freely interconvertible.
- Calling sync, blocking code from inside an async context is a *bug* — it stalls the executor. Use `tokio::task::spawn_blocking` to isolate blocking calls to a dedicated thread pool.
- Calling async code from sync is possible via `block_on`, but then you are paying for an executor setup.

The rule of thumb: pick one colour for the main body of your application, and use the escape hatches (`spawn_blocking`, `block_on`) sparingly at the boundary.

8.8 Threads vs async

Question

Use threads

Use async

Is the work CPU-bound?

Yes

No

Is the work I/O-bound?

Maybe

Yes, especially at scale

Do I need thousands of concurrent operations?

No

Yes

Do I want independence between operations?

Yes

Either

Is my ecosystem async-friendly?

Doesn't matter

Check

Do I need a simple mental model?

Threads are simpler

Async is more indirect

Many real applications use both: a tokio runtime handling tens of thousands of network connections, with `spawn_blocking` or a separate thread pool (often via `rayon`) for the CPU-heavy work.

8.9 What you get

The compiler does not stop you from deadlocking yourself. It does not stop you from writing a tokio task that busy-loops and starves the executor. It does not stop you from ordering your log messages wrong.

What it gives you, instead, is a much narrower cone of uncertainty:

- You will not read memory that another thread just freed.
- You will not write memory another thread is reading.
- You will not share a non-thread-safe type between threads.
- You will not forget to release a lock after an early return.
- You will not await a future in a place where awaiting is meaningless.

Every one of those was, until Rust, a bug class that systems programmers fought for their entire careers. Now they are just compile errors. The chapter — and this book — ends there, because that is the whole promise made good. Everything beyond here is the rest of your career.

End of book.